



DOWNBELOW

WITH DR. DIDI

INTERNAL WEBSITE OPERATING GUIDE

DownBelow Website Zero to Hero

A Straightforward Guide to Understanding and
Running the DownBelow Website Like the
Engineers That Built It

Made for non-technical mobile operators

01

Know what becomes
public

02

Run every feature
confidently

03

Avoid mistakes and
troubleshoot cleanly

PREPARED BY

Bespoke Technologies

Engineering the solutions for this, and the Next Generations_





MOBILE-FIRST INTERNAL TEAM GUIDE

DownBelow Website Zero to Hero

A Straightforward Guide to Understanding and Running the DownBelow Website Like the Engineers That Built It

For: DownBelow Family Health Initiative internal team

Prepared by: Bespoke Technologies

Edition: June 3, 2026

Designed for mobile reading: single-column, clear action cards, large text, and practical operating steps.

A Special Note from our Team Leader

Dear DownBelow Family,

This website exists for one purpose - to amplify the meaningful work you do every day.

We built it to be powerful, secure, and easy to manage. Whether you are posting an article, adding an event, updating a program, replying to V-Vault, or helping someone find information, every action online has real-world impact.

This guide is your companion. It is here to help you understand how the website works, avoid costly mistakes, and solve common problems quickly.

You do not need to be a developer to use the platform well. You need care, consistency, and the confidence to follow the right steps.

Thank you for being the heartbeat behind DownBelow.

With gratitude,

Bespoke Tech

Bespoke Technologies

Engineering the solutions for this, and the Next Generations_

Start Here: How to Use This Guide on a Phone

Operator note

This edition is designed for scrolling, quick decisions, and simple operating steps. You do not need to understand code before using it.

Read this first

Start with the mental model, then jump to the module you need. Do not read the technical appendices first unless you are maintaining the website.

Use the action steps

Every feature has a practical operating path. Follow the steps in order, then test the public page after saving.

Protect private information

If something contains private health details, identity information, passwords, access codes, or internal notes, do not put it in public content fields.

Escalate when needed

If login, media upload, database, email, or V-Vault response flows fail repeatedly, ask the technical maintainer to inspect the route, API, repository, database, storage, and email provider.

The five rules to remember

Draft first

Create content as draft. Publish only after review.

Public means public

Anything saved in a public field can be seen by visitors once published.

Archive before deleting

Archive when history matters. Delete only when necessary and authorized.

Use the lowest role

Not everyone should be a top-level admin. Give people only the access they need.

Test after saving

Open the public page after important admin changes. Do not assume it worked.

Copyright and Disclaimer

This internal guide is prepared for DownBelow Family Health Initiative. It is based on the repository state inspected on the date shown on the title page. It is operational documentation, not legal, medical, security, or financial advice.

The book intentionally does not include private credentials, real secrets, database passwords, API keys, or access codes. Use your password manager, deployment provider, and internal security process for those values.

Warning

If the website code changes after this edition, use the current repository as the source of truth and update this guide before relying on changed flows.

Dedication

For the DownBelow team members who answer sensitive questions with care, protect trust, and keep the public platform useful for families, individuals, and communities.

Acknowledgements

This guide reflects the work embedded in the DownBelow website: public education pages, private question workflows, admin controls, content moderation, email flows, media storage, audit logging, and database-backed operations.

Special acknowledgement goes to the operators who will use the system daily. Good software only becomes useful when people run it with discipline.

Preface

This book teaches the website from first principles. A website is not just pages. It is a public experience, an admin control plane, a database, an email system, file storage, security rules, and operating habits. If one part is misunderstood, the whole system feels mysterious.

The goal is not to turn every operator into a full-time developer. The goal is to make the system legible enough that the team can operate it confidently, avoid avoidable mistakes, and know when to escalate to an engineer.

Who This Book Is For

Non-technical operators

Understand where features live, how to use admin tools, and what not to touch.

Content editors

Publish articles, events, gallery images, reviews, podcast episodes, team profiles, and alerts safely.

Moderators

Recognize comments, reviews, V-Vault submissions, and user-account issues that need attention.

Top-level admins

Manage users, admin accounts, sensitive V-Vault identity, settings, deletion, health checks, and production readiness.

Developers and maintainers

Find the real source files, data models, environment variables, and operational contracts.

How to Use This Book

Read Part I once before operating the site. It gives the mental model. Use Part II when working on the public website. Use Part III when running the admin side. Use Part IV before deployment or debugging infrastructure. Use Part V when you need a checklist or quick answer.

Tip

For daily work, start with the Operator Quick Reference appendix. For production changes, use the deployment checklist and the environment-variable appendix.

Table of Contents

Front Matter

Title Page; Copyright and Disclaimer; Dedication; Acknowledgements; Preface; Who This Book Is For; How to Use This Book

Part I - The Operating Model

Architecture mental model, public/admin/backend map, role model, source truth

Part II - Running the Public Website

Home, alerts, PWA, About, Team, Outreach, Contact, Events, Library, Podcast, Gallery, Reviews, V-Vault, public accounts

Part III - Running the Admin Side

Admin auth, dashboard, platform health, settings, alerts, media, boards, user management, admin accounts, V-Vault moderation, audit logs

Part IV - Backend and Production Operations

Next.js App Router, Prisma/CockroachDB, migrations, R2, Resend, env, rate limiting, fallback behavior, security, deployment

Part V - Checklists and Troubleshooting

Daily/weekly/release checklists, troubleshooting table, glossary, appendices, operator quick reference

Executive Summary

The DownBelow website is a public education and engagement platform with an internal admin control plane. The public side helps people learn, register, contact the team, follow events, submit reviews, ask private V-Vault questions, and receive account notifications. The admin side lets approved team members manage content, media, users, admins, alerts, and sensitive moderation workflows.

The highest-leverage operating rule is simple: know what becomes public, know which role can change it, and know which data is saved. Most mistakes happen when a team member treats a public field like a private note, publishes before review, deletes instead of archiving, or assumes an email/notification was sent without checking the actual flow.

The current codebase uses Next.js App Router, Prisma with CockroachDB, Cloudflare R2 for media, Resend for transactional email, role-based admin access, audit logging, validation schemas, database-backed rate limiting with a memory fallback, structured API errors with request IDs, and public database fallback for selected read paths. Admin writes are intentionally stricter and should fail loudly when core services are unavailable.

The newest operations layer is the top-level-admin Platform Health page. It does not replace engineering logs, but it gives operators one place to check database reachability, email configuration, media storage configuration, admin environment readiness, route groups, model probes, and the error-reference table before guessing why a page or API failed.

Public and admin sessions are separate.

A public login does not grant admin access; admin APIs verify admin account state.

Status controls public visibility.

Draft is private to admin workflows; published is public; archived preserves history while hiding content.

R2 stores files; the database stores references.

Deleting files outside the app can break public pages.

Email verification matters.

Users and admins can be blocked until verification is completed.

V-Vault is private, not technically untraceable.

Identity is protected publicly but can be revealed by authorized top-level admins and audited.

Some public pages fall back when the database is unavailable.

Old/static content during an outage is not proof that records disappeared.

Introduction

The DownBelow website has two sides: the public side and the admin side. The public side is what visitors see. The admin side is where trusted team members manage content, users, sensitive questions, media, alerts, and settings. Behind both sides are shared services: Next.js routes, Prisma models, CockroachDB records, Cloudflare R2 assets, Resend emails, session cookies, rate limits, validation schemas, and audit logs.

The conventional answer is: learn each page and form. The stronger strategic answer is: learn the flow of data. Once you understand what is saved, who can change it, and what becomes public, every feature becomes easier to operate.

Part I - The Operating Model

Chapter 1 - The Mental Model

DownBelow runs like a small publishing and care platform. Public pages educate and build trust. Authenticated users can participate more deeply through comments, reviews, account profile, and V-Vault. Admins control the content and sensitive workflows. The database remembers the state. Email helps verify identity and recover access. R2 stores large files.

Public website

What it does: Shows public content and intake forms.

What breaks when misunderstood: Private or draft content may be treated as public by mistake.

Admin control plane

What it does: Lets trusted staff manage records.

What breaks when misunderstood: Wrong roles can cause failed saves or overexposed permissions.

Database

What it does: Stores users, content, moderation, settings, audit logs.

What breaks when misunderstood: Outages change public fallback behavior and block admin writes.

Email

What it does: Sends verification and reset messages.

What breaks when misunderstood: Users/admins cannot complete identity flows if provider is missing.

Media storage

What it does: Stores large files outside the database.

What breaks when misunderstood: Broken URLs or deleted files break public pages.

Security rules

What it does: Protect sessions, roles, rate limits, and private data.

What breaks when misunderstood: Bypassing them creates account and privacy risk.

Do not do this

Do not treat admin pages as a private notebook. Anything saved in a public content field can become public immediately or after publishing.

Chapter 2 - Source Truth and Route Map

This guide was written after inspecting the current App Router routes, admin components, Prisma schema, validation schemas, session logic, email templates, R2 storage code, public fallback logic, and environment configuration. The coverage checklist in the appendix lists the inspected surfaces.

The current working tree does not contain active admin recovery/ reactivation routes that older notes may mention. Admin recovery in the current route inventory is handled through admin forgot password, reset password, verify email, and admin account activation/ deactivation.

The current working tree does contain an active Platform Health surface at `/admin/health` and `/api/admin/health`. It is top-level-admin-only in practice because `founder_admin` and `super_admin` share the top RBAC rank. It reads health signals from `src/lib/admin/health.ts`, checks configured dependencies and Prisma model probes, and shows route-level causes and operator actions when a dependency is blocked or degraded.

Warning

If someone asks for `/admin/recovery`, verify the current route inventory first. In this inspected repository state, that route is deleted/not active.

Chapter 3 - Roles and Responsibility

RBAC means role-based access control. DownBelow uses role rank: moderator, editor, `founder_admin`, `super_admin`. A higher rank can do what lower ranks can do when the API checks the required role. In the current code, `founder_admin` and `super_admin` share the same top-level rank; the name is different, but their route authority is equivalent where `canAccessRole` is used.

moderator

Rank: 1

Typical work: View many admin records; moderate event comments where allowed.

Sensitive limits: Cannot publish most content or manage accounts.

editor

Rank: 2

Typical work: Create/update content, manage alerts, upload media.

Sensitive limits: Cannot delete many records or manage users/admins.

founder_admin

Rank: 4

Typical work: Top-level founder authority, library/editorial leadership, sensitive workflows, and any route guarded by super_admin rank.

Sensitive limits: Must be protected like super_admin because the current RBAC rank is equal.

super_admin

Rank: 4

Typical work: Top-level operations authority for settings, users, admin accounts, V-Vault, deletion, health checks, and sensitive workflows.

Sensitive limits: Must be kept rare and protected.

Part II - Running the Public Website

SEO, Robots, Sitemap, Metadata, Structured Data, and Legal Pages

Mental model

SEO is not a magic traffic switch. It is the site identity card for machines. It works best when public content is accurate, crawlable, and consistently named.

What it is

The search and installability layer that tells browsers, crawlers, and social platforms what the site is, what can be indexed, and how public pages should appear when shared.

Who uses it

Visitors benefit from it indirectly; operators and maintainers verify it before launches; search engines and social platforms consume it.

Where it lives

`src/app/layout.tsx`, `src/app/robots.ts`, `src/app/sitemap.ts`, `src/app/manifest.ts`, `StructuredData.tsx`, `/privacy`, `/terms`.

What the public sees

Page titles, share previews, icons, install metadata, privacy page, terms page, footer legal links, /me account legal links, and search-result snippets.

What admins control

Site settings influence public brand/contact values. Most SEO structure is code-managed.

Data saved

SEO metadata is mostly code/config. Sitemap pulls published articles, gallery images, and events from the database when available.

Emails or notifications

No emails or notifications are triggered by SEO features.

Source truth

src/app/layout.tsx; src/app/robots.ts; src/app/sitemap.ts; src/app/manifest.ts; src/components/seo/StructuredData.tsx; src/lib/site-config.ts; privacy/terms pages.

How to operate it

1. Before launch, confirm NEXT_PUBLIC_SITE_URL is the production domain.

2. Open `/robots.txt` and confirm `/admin/` is disallowed while public pages are allowed.
3. Open `/sitemap.xml` and confirm important public URLs appear.
4. Check a public article, event, gallery image, and podcast page title/meta in the browser.
5. Review `/privacy` and `/terms` after major workflow changes, especially auth, reviews, V-Vault, notifications, or contact intake.
6. Confirm the links remain reachable from the footer and from the `/me` account area.

Common mistakes and fixes

Wrong production site URL

What happens: Emails, metadata, sitemap, and canonical links point to the wrong host.

Fix: Fix `NEXT_PUBLIC_SITE_URL` in production env and redeploy.

Trying to index admin pages

What happens: Private admin surfaces can appear in crawler paths.

Fix: Keep robots disallow for `/admin/` and never link admin URLs publicly.

Ignoring legal pages after workflow changes

What happens: Privacy/terms may no longer match actual data flows.

Fix: Update legal copy whenever data collection changes.

Security

Do not expose admin URLs, private tokens, unpublished content, or private media in metadata, sitemap, structured data, or public legal examples.

Troubleshooting

Search preview looks old

Likely cause: Crawler cache or stale metadata.

First response: Verify current metadata, then request recrawl from platform tools if needed.

Sitemap missing content

Likely cause: Content is draft/archived or database unavailable.

First response: Confirm status and database health.

PWA icon missing

Likely cause: Manifest icon path missing or file not deployed.

First response: Check `/manifest.webmanifest`, `/icon-192.png`, and `/icon-512.png`.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Admin routes are intentionally blocked from crawling.

Sitemap and metadata depend on correct production base URL. |
Exact metadata and structured data fields in layout, site-config,
and StructuredData component.

Home Page, Navigation, Site Alerts, and PWA Install Prompt

Mental model

Think of the home page as reception. It should orient people quickly, send them to the right room, and show urgent notices without blocking the rest of the work.

What it is

The public front door of the website. It introduces DownBelow, links people to library articles, V-Vault, events, contact, reviews, team, gallery, and podcast content, and shows the active site alert ticker on the home route.

Who uses it

Visitors, returning members, staff, and admins checking public presentation.

Where it lives

Public routes / and /home where /home redirects to /; components in src/app/page.tsx, Navbar.tsx, SiteAlertTicker.tsx, InstallPrompt.tsx.

What the public sees

Hero copy, current hero image, pillar sections, V-Vault preview, latest articles, navigation, install prompt after repeat visits, and active notices on the home page.

What admins control

Site settings control the hero and footer copy. Site alerts control the ticker text, speed, duration, schedule, and active state.

Data saved

SiteSettings and SiteAlert records. The install prompt only stores browser localStorage dismissal and visit-count flags.

Emails or notifications

No email is sent from the home page. Alert dismissal is local to the browser.

Source truth

`src/app/page.tsx`; `src/app/home/page.tsx`; `src/components/layout/Navbar.tsx`; `src/components/layout/SiteAlertTicker.tsx`; `src/components/layout/InstallPrompt.tsx`; `src/app/api/alerts/active/route.ts`.

How to operate it

1. To update the headline, body, or hero image, sign in to `/admin`, open Site Settings, edit the fields, upload or paste the final image URL, and save.

2. To publish a notice, open Site Alerts, create a short plain-language alert, set start and end dates, keep it active, and test the home page in a private browser.
3. To remove a notice from public view, either turn off Active or set an end time in the past.
4. To test the PWA prompt, use a supported browser, visit the public site more than once, wait at least 15 seconds, and confirm the prompt is not shown on admin pages.

Common mistakes and fixes

Writing a very long alert

What happens: The ticker becomes hard to scan and can feel noisy.

Fix: Keep alerts short: one instruction, one deadline, one call to action.

Expecting alerts everywhere

What happens: The current ticker only displays on / and /home.

Fix: Use the home page as the notice surface unless code is changed.

Using a decorative hero image with no alt text

What happens: The site becomes less accessible and less trustworthy.

Fix: Use meaningful images and alt text that describes the image purpose.

Security

Do not put private phone numbers, private health details, admin links, tokens, or internal incident notes in public alerts or site settings.

Troubleshooting

Alert not visible

Likely cause: Inactive alert, future start date, expired end date, or dismissed in localStorage.

First response: Check the alert schedule, test in an incognito window, then inspect `/api/alerts/active`.

Install prompt not visible

Likely cause: Browser did not fire `beforeinstallprompt`, user dismissed it, or first visit threshold not met.

First response: Test in Chrome/Edge, clear localStorage, and verify `/sw.js` is reachable.

Hero copy did not update

Likely cause: Settings save failed or public fallback is being used.

First response: Check admin save response and database availability.

Knowledge split

Must know by heart

Must recognize | Lookup-only

The home route is `/` ; `/home` redirects to `/` . Alerts only show on the home experience.

Install prompt state is browser-local, not stored in the database. |
Exact alert animation math and localStorage keys in
SiteAlertTicker.tsx and InstallPrompt.tsx.

About, Team, and Outreach Pages

Mental model

These pages are the credibility layer. They answer: who are you, why should people trust you, and what work is already happening?

What it is

Public trust-building pages that explain the mission, introduce the people behind DownBelow, and show outreach activity.

Who uses it

Visitors, partners, prospective volunteers, and staff preparing public information.

Where it lives

`/about`, `/team`, and `/outreach`.

What the public sees

Mission, vision, Dr. Didi introduction, team member cards, outreach metrics, gallery-style activity sections, and calls to contact or volunteer.

What admins control

Team members are managed in Admin Team. Outreach page content currently comes from static data files, not an admin board.

Data saved

TeamMember records for database-backed team profiles. Outreach page data is static project content.

Emails or notifications

No email or notification is triggered by viewing these pages.

Source truth

`src/app/about/page.tsx`; `src/app/team/page.tsx`; `src/app/outreach/page.tsx`; `src/components/admin/TeamMembersBoard.tsx`; prisma TeamMember model.

How to operate it

1. Open `/admin/team` to add or update a team member.
2. Upload a clear image, write useful alt text, select the correct tier, and publish only when the profile is complete.

3. Use draft status for incomplete profiles and archived status for people who should no longer appear publicly.
4. Review /about and /team after changes because both pages can display team information.

Common mistakes and fixes

Publishing incomplete bios

What happens: The public page looks unfinished and weakens trust.

Fix: Keep incomplete people as draft until image, role, tier, and bio are ready.

Wrong team tier

What happens: A person appears in the wrong section.

Fix: Use founder for Dr. Didi, leadership for lead operators, core for supporting team.

Expecting Outreach to be editable in admin

What happens: No admin changes appear because the page uses static content.

Fix: Treat Outreach as a developer/content update unless an admin module is added.

Security

Only publish team details the person has approved. Do not expose private numbers, private email addresses, or internal role notes in public bios.

Troubleshooting

Team member missing

Likely cause: Status is not published, sort order is unexpected, or database fallback is active.

First response: Check status, tier, and the Team admin list.

Image missing

Likely cause: Media URL is invalid or deleted.

First response: Confirm the asset still exists in R2 and is not a local-only broken path.

Outreach content outdated

Likely cause: Static data needs a code/content update.

First response: Ask a maintainer to update the outreach data source.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Team is admin-managed; Outreach is currently static.

The same team records can support more than one public page. |
Exact TeamMember fields and tiers in Prisma and
TeamMembersBoard.tsx.

Contact Page and Contact Requests

Mental model

Contact is an intake tray. It collects structured requests so the team can follow up with less confusion than a free-form social DM.

What it is

The public request form for people who want to reach DownBelow, request support, or book a conversation.

Who uses it

Visitors submit it; staff monitors contact submissions and official email/WhatsApp.

Where it lives

/contact and /api/contact.

What the public sees

Contact form fields for name, email, optional Nigerian phone, preferred date, preferred time, and message.

What admins control

There is currently no dedicated Contact admin board in the route inventory. ContactSubmission records exist in the database and dashboard counts include them.

Data saved

ContactSubmission: first name, last name, email, phone, preferred date/time, message, status, notes, timestamps.

Emails or notifications

The current `/api/contact` route saves the request. It does not send a Resend email.

Source truth

`src/app/contact/page.tsx`; `src/components/contact/ContactForm.tsx`; `src/app/api/contact/route.ts`; prisma ContactSubmission model.

How to operate it

1. Ask the person to use `/contact` when the team needs structured details.
2. Review the submitted details from the database or any future admin surface connected to ContactSubmission.
3. Follow up through the official channels in the site settings: contact email or WhatsApp.

4. If the request is sensitive, move it into the right private workflow instead of discussing it publicly.

Common mistakes and fixes

Assuming email is sent automatically

What happens: A request may sit in the database without an inbox copy.

Fix: Use database/admin monitoring until contact email notification is added.

Putting clinical details into alerts or comments

What happens: Private information may become public.

Fix: Direct users to contact or V-Vault depending on sensitivity.

Ignoring rate limits

What happens: Repeated attempts from the same IP can be blocked.

Fix: Wait or collect the request through an official manual channel.

Security

Contact messages can contain sensitive health context. Treat them as private operational records and do not paste them into public admin notes.

Troubleshooting

Form says request failed

Likely cause: Validation error, rate limit, or database issue.

First response: Check required fields, wait if rate limited, then inspect server logs.

Preferred time rejected

Likely cause: Only listed time windows are accepted in the UI.

First response: Use an available time or follow up manually.

No email arrived

Likely cause: No email send is implemented in the contact API.

First response: Look for the saved ContactSubmission record.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Contact saves a database record; it is not currently an email notification system.

Contact requests are different from V-Vault questions. | Validation rules in contactSchema and storage model in Prisma.

Events, Event Detail Pages, Likes, Comments, and Streams

Mental model

An event is a public program record. The listing helps people discover it; the detail page is the operating page during and after the event.

What it is

Published outreach or online events with detail pages, optional live stream embedding, authenticated likes, and authenticated comments.

Who uses it

Visitors browse; registered users like and comment; admins create events and moderate comments.

Where it lives

/events, /events/[slug], /api/events/*, Admin Events.

What the public sees

Event cards, event detail pages, cover images, schedules, location/community labels, stream embeds, like button, comments, and share menu.

What admins control

Event title, slug, summary, body, cover image, stream URL/provider, live flag, engagement flag, status, publish date, schedule, sort order, and comment status.

Data saved

OutreachEvent, EventLike, EventComment records. Likes are unique per event and user. Comments default to visible.

Emails or notifications

No email notification is triggered by likes or comments in the current code.

Source truth

`src/app/events/; src/components/events/; src/lib/events/repository.ts; src/lib/events/schemas.ts; src/components/admin/EventsBoard.tsx; prisma OutreachEvent/EventLike/EventComment.`

How to operate it

1. In `/admin/events`, create a draft event with a clear title, slug, summary, body, schedule, and cover image.
2. Set stream provider and URL only when the stream is ready. Use the live flag when the event is actually live.
3. Publish the event by setting status to published and a publish date.

4. After publishing, open /events and the detail page to confirm the image, metadata, and engagement controls.
5. Moderate comments from the admin event comments panel by setting visible, hidden, or flagged.

Common mistakes and fixes

Turning on live too early

What happens: Visitors expect a live session before it starts.

Fix: Use live only when the stream should be treated as live.

Disabling engagement unintentionally

What happens: Likes/comments become unavailable.

Fix: Check engagementEnabled before publishing.

Bad slug changes after sharing

What happens: Old links break or show not found.

Fix: Choose a stable slug before promotion.

Security

Only authenticated public users can like and comment. Admin moderation should hide or flag inappropriate comments without exposing private user details publicly.

Troubleshooting

Like asks for login

Likely cause: User is not authenticated.

First response: Ask the user to log in or register first.

Comments fail quickly

Likely cause: Per-user event comment rate limit may be hit.

First response: Wait at least a minute and avoid repeated submissions.

Stream does not render

Likely cause: Unsupported or malformed stream URL/provider.

First response: Check YouTube/Facebook URL formatting and provider selection.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Events are public only when status is published. Likes/comments require login.

Comments are visible by default but can be hidden or flagged by admins. | Event schema fields and stream utility handling in src/lib/events.

Health Library and Article Detail Pages

Mental model

The library is the knowledge base. It turns repeated education into reusable public content, reducing one-on-one explanation load for the team.

What it is

The educational article library with category filtering, search, article pages, related articles, and medical disclaimers.

Who uses it

Visitors learn from it; top-level admins manage it.

Where it lives

/library, /library/[slug], /api/admin/library/*.

What the public sees

Article cards, search, category filters, cover images, read time, author, content blocks, related articles, and disclaimers.

What admins control

Article title, slug, summary, body, topic/category, cover image, author, read time, status, publish date, and sort order.

Data saved

Article records with publication status and metadata. Static seed articles are used as fallback if database content is unavailable or empty.

Emails or notifications

No email or notification is triggered by article publishing.

Source truth

`src/app/library/*`; `src/lib/library/repository.ts`; `src/components/admin/LibraryArticlesBoard.tsx`; prisma Article model.

How to operate it

1. Prepare the article content with a clear title, short summary, category, and body.
2. Open Admin Library, create a draft, and add a stable slug before promotion.
3. Upload or choose a cover image with useful alt text.
4. Publish only after reviewing the medical disclaimer context and public page layout.
5. Use archived for content that should no longer appear publicly but should remain in records.

Common mistakes and fixes

Publishing unreviewed health advice

What happens: Users may act on incomplete information.

Fix: Have qualified review before publishing sensitive medical content.

Changing slugs after social sharing

What happens: Shared links can break.

Fix: Use redirects or avoid slug changes after publishing.

Assuming DB outage means no library

What happens: Public pages can fall back to static articles.

Fix: Understand the fallback before treating it as data loss.

Security

The library educates. It must not replace diagnosis, emergency care, or private consultation. Keep personal case details out of articles unless fully anonymized and approved.

Troubleshooting

Article missing publicly

Likely cause: Draft/archived status, unpublished date, or database fallback.

First response: Check status and publishedAt, then test route by slug.

Search does not find article

Likely cause: Search looks at current public article data.

First response: Confirm the article is published and contains the terms.

Cover image broken

Likely cause: Invalid URL or deleted asset.

First response: Reattach an existing media asset or upload a new one.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Only published articles belong on the public library.

Static fallback articles can appear when database content is unavailable. | Article data mapping, fallback behavior, and read-time logic in library/repository.ts.

Podcast Listing and Podcast Detail Pages

Mental model

Podcast episodes are structured media records. The database stores the episode metadata; R2 stores the heavy audio and image files.

What it is

A public audio content section with episode listing, detail pages, audio controls, download links, show notes, transcripts, guests, and topic tags.

Who uses it

Visitors listen; editors create and update episodes; top-level admins delete.

Where it lives

/podcast, /podcast/[slug], /api/admin/podcast/*.

What the public sees

Episode cards, cover art, audio player, duration, guest, topic tags, download button, source link, show notes, transcript, and related episodes.

What admins control

Episode title, slug, summary, description/show notes, audio upload URL, audio size/type, cover art, guest, tags, transcript, external source, status, publish date, and sort order.

Data saved

PodcastEpisode records. Audio and cover assets are usually stored in R2 and referenced by URL.

Emails or notifications

No email or notification is triggered by podcast publishing.

Source truth

src/app/podcast/*; src/components/admin/PodcastEpisodesBoard.tsx; prisma PodcastEpisode model; media upload APIs.

How to operate it

1. Upload the audio file through the admin podcast form or media upload flow.
2. Enter a stable slug, summary, guest name if relevant, tags, show notes, and transcript when available.
3. Upload cover art or use a consistent public cover image.
4. Set status to published and verify the player works on /podcast and /podcast/[slug].
5. Use external source URL only for official mirrors or source platforms.

Common mistakes and fixes

Uploading unsupported audio

What happens: Validation blocks the upload.

Fix: Use mp3, mp4 audio, webm, ogg, or wav within the allowed size.

No transcript for sensitive topics

What happens: Accessibility and search quality suffer.

Fix: Add transcripts for important episodes whenever possible.

Deleting audio still used by an episode

What happens: Delete is blocked by usage protection.

Fix: Remove or replace the episode reference before deleting the media asset.

Security

Review episode content before publishing. Do not publish identifying health stories without consent.

Troubleshooting

Audio will not upload

Likely cause: File too large, unsupported type, R2 CORS issue, or presigned upload failed.

First response: Check file type/size and R2 CORS PUT settings.

Episode missing publicly

Likely cause: Draft/archived status or publish date issue.

First response: Check status and publishedAt.

Download link fails

Likely cause: Audio URL or R2 public URL is wrong.

First response: Open the asset URL directly and replace if needed.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Audio lives in storage; episode metadata lives in the database.

The podcast board uses the same media upload infrastructure as other admin modules. | Allowed media types and size limits in media-policy.ts.

Gallery Listing, Image Detail Redirects, and Image Modal

Mental model

The gallery is visual evidence of the work. Each image should be easy to understand without staff standing beside the viewer.

What it is

A public gallery grid with categories and an image detail experience powered by query parameters and modal navigation.

Who uses it

Visitors browse; editors publish images; top-level admins delete records.

Where it lives

/gallery, /gallery/[slug], /api/gallery/*, Admin Gallery.

What the public sees

Category filters, image cards, modal view, title, caption, description, event name, location, capture date, and keyboard/swipe navigation.

What admins control

Image title, slug, image URL/upload, alt text, category, status, event name, location, captured date, sort order, caption, and description.

Data saved

GalleryImage records. Public gallery can fall back to static gallery data if database content is unavailable.

Emails or notifications

No email or notification is triggered.

Source truth

src/app/gallery/*; src/components/content/ImageViewModal.tsx; src/components/admin/GalleryImagesBoard.tsx; prisma GalleryImage model.

How to operate it

1. Upload a high-quality image through Admin Gallery or Media.
2. Write useful alt text, title, caption, and description.
3. Assign a category and optional event/location metadata.
4. Publish the image and open /gallery to verify the card and modal.
5. Use archived for images that should stop appearing publicly without deleting their record.

Common mistakes and fixes

Missing alt text

What happens: Accessibility and search quality suffer.

Fix: Describe the meaningful content of the image.

Publishing low-quality images

What happens: The public record of the work looks careless.

Fix: Use clear, well-lit, relevant images.

Deleting shared image assets

What happens: Deletion may be blocked if used, or broken if removed outside the app.

Fix: Replace references before deleting media.

Security

Do not publish images that reveal private health information, minors, or identifiable participants without proper consent.

Troubleshooting

Image detail route redirects

Likely cause: `/gallery/[slug]` intentionally redirects to `/gallery?image=slug`.

First response: Use the gallery modal URL as the public detail link.

Image missing

Likely cause: Not published, wrong category filter, or fallback data active.

First response: Check status, category, and database health.

Modal navigation fails

Likely cause: Browser/client script issue.

First response: Reload, test keyboard Escape/arrow controls, and inspect console errors.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Gallery detail pages redirect into the gallery modal.

Category and status determine public visibility. | Exact modal interaction behavior in ImageViewModal.tsx.

Reviews Page, Public Submissions, Helpful Marks, and Team Replies

Mental model

Reviews are public trust signals. They must be sincere, consented, and moderated so they support confidence without exposing private details.

What it is

A public testimonial/review page with star ratings, public submissions, helpful counts, and admin replies.

Who uses it

Visitors read and submit; admins moderate, create, edit, reply, archive, and delete.

Where it lives

/review, /api/reviews, /api/reviews/[id]/helpful, Admin Reviews.

What the public sees

Published reviews, average rating, helpful counts, review form, consent checkbox, and optional team reply.

What admins control

Review name, role/context, location, rating, body, status, source, sort order, published date, admin reply, reply author, and reply timestamp.

Data saved

Review and ReviewHelpful records. Helpful marks are unique by authenticated user or anonymous visitor cookie.

Emails or notifications

No email is sent when reviews are submitted or marked helpful.

Source truth

`src/app/review/page.tsx`; `src/components/reviews/*`; `src/lib/reviews/repository.ts`; `src/components/admin/ReviewsBoard.tsx`; `prisma Review/ReviewHelpful`.

How to operate it

1. Ask public submitters to confirm consent before publishing a review.
2. Review new public submissions in Admin Reviews because current public submissions are created as published.

3. Edit or archive anything that contains private medical details, defamation, or unsafe claims.
4. Add a team reply when clarification or appreciation helps future readers.
5. Use helpful counts as a signal, not as medical quality proof.

Common mistakes and fixes

Assuming public submissions wait for approval

What happens: Current API saves them as published.

Fix: Monitor reviews after public submissions and archive unsafe content quickly.

Leaving private details in a review

What happens: A person may unintentionally disclose sensitive information.

Fix: Edit/archive and contact the person privately if needed.

Treating helpful marks as votes from unique people only

What happens: Anonymous visitor cookies are browser-based.

Fix: Use helpful count as lightweight feedback.

Security

Never publish private health information or identifying stories without explicit consent. Reviews are public content.

Troubleshooting

Helpful button does nothing

Likely cause: Seed fallback review or cookie/session issue.

First response: Test with a database review and clear visitor cookie if needed.

Review form rejected

Likely cause: Consent missing, rating invalid, or text too short/long.

First response: Review form validation messages.

Review not visible

Likely cause: Archived/draft status or fallback list active.

First response: Check status in Admin Reviews.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Public review submission currently publishes immediately.

Team replies are public and should read like official DownBelow communication. | Review source/status rules in reviews/repository.ts and Prisma.

Part III - Running the Admin Side

V-Vault Anonymous/Public Question Flow and Private Responses

Mental model

V-Vault is a protected intake and response workflow. Public identity is hidden from the crowd, but the system links submissions to a user account so the team can reply privately.

What it is

A private question submission flow for sensitive health questions, connected to authenticated public users and handled through a role-split admin board.

Who uses it

Registered users submit questions. Moderators can view masked submissions, editors and higher can moderate/respond, and top-level admins can reveal identity when needed.

Where it lives

/vault, /api/vault, /api/vault/me, Admin V-Vault.

What the public sees

V-Vault introduction, login/register prompt if signed out, category picker, question form, support contact guidance, and FAQ preview.

What admins control

Submission status, moderation notes, FAQ title, private response text, and identity viewing for authorized roles.

Data saved

VaultSubmission, VaultResponse, VaultSubmissionEvent, and UserNotification records.

Emails or notifications

When an admin creates a response, the system creates an in-app user notification. The email template exists, but the current response repository creates in-app notification only.

Source truth

`src/app/vault/page.tsx`; `src/components/vault/VaultForm.tsx`; `src/app/api/vault/`; `src/lib/vault/public-preview.ts`; `src/components/admin/VaultModerationBoard.tsx`; *prisma Vault models.*

How to operate it

1. Make sure V-Vault submissions are enabled with `VAULT_SUBMISSIONS_ENABLED=true` before inviting users to submit.
2. A user registers, verifies email, signs in, opens `/vault`, chooses a category, and writes a 50-500 character question.
3. A moderator can open Admin V-Vault and view masked submissions.

4. An editor or higher updates status/moderation notes and writes a private response when appropriate.
5. A top-level admin reveals identity only when necessary and with care.
6. The user receives an in-app notification and can read the response in /me under V-Vault notifications/history.

Common mistakes and fixes

Calling it fully anonymous internally

What happens: Admins with the right role can link submissions to users when needed.

Fix: Explain it as publicly anonymous/private, not technically untraceable.

Letting low-trust roles reveal V-Vault identity

What happens: Private user identity and sensitive health context can be exposed beyond the approved top-level roles.

Fix: Keep identity reveal to top-level admins. Editors may moderate/respond, but should not receive identity access unless their role is elevated deliberately.

Expecting email on response

What happens: Current code creates in-app notification, not response email.

Fix: Tell users to check /me; add email sending only if implemented.

Security

Treat V-Vault as sensitive. Identity viewing is restricted to top-level admins and is audit-logged. Do not export or screenshot identifiable question data casually.

Troubleshooting

Vault form unavailable

Likely cause: Feature flag disabled or user not signed in.

First response: Check VAULT_SUBMISSIONS_ENABLED and session state.

User cannot see response

Likely cause: Notification unread state, wrong account, or no response record.

First response: Check /me, then verify the VaultSubmission is linked to that user.

Admin cannot access Vault

Likely cause: Role below moderator for viewing, below editor for moderation/response, or below top-level admin for identity reveal.

First response: Match the role to the action: moderator for masked viewing, editor+ for response/moderation, top-level admin for identity reveal.

Knowledge split

Must know by heart

Must recognize | Lookup-only

V-Vault is private on the public side; masked viewing, response, and identity reveal have different admin thresholds.

Private response creates an in-app notification and event history. |
Identity reveal auditing and status handling in admin repository.

Public Authentication, Account Area, and Notifications

Mental model

Authentication proves who is using sensitive features. Verification protects the person and the platform from fake or mistyped accounts.

What it is

The public user account system for registration, email verification, login, logout, forgot/reset password, profile management, password changes, and notifications.

Who uses it

Public users, members, and staff helping users regain access.

Where it lives

/register, /login, /forgot-password, /reset-password, /verify-email, /me, /api/auth/, /api/users/.

What the public sees

Register/login forms, email verification page, reset forms, profile editor, password change area, logout action, and V-Vault notification/history widgets.

What admins control

Top-level admins manage users through Admin User Management. Admins do not directly know user passwords.

Data saved

User records with email, display name, password hash, phone, active state, email verification tokens, reset tokens, failed attempts, lockout, last activity, notifications, and V-Vault links.

Emails or notifications

Registration sends verify email. Resend verification endpoint sends a new verify email. Forgot password sends reset email. Reset/change password sends password-changed email.

Source truth

`src/components/auth/; src/app/api/auth/; src/app/me/page.tsx;
src/lib/auth/session.ts; src/lib/admin/user-repository.ts; src/lib/
email/templates.ts.`

How to operate it

1. A user registers with email, display name, optional Nigerian phone, and password.
2. The user must verify email before login succeeds.
3. If a user forgets the password, send them to `/forgot-password`; the response stays generic for security.
4. The reset link opens `/reset-password?token=...` and expires after one hour.
5. After login, the user can update profile and password in `/me`; V-Vault responses appear in `notifications/history`.

Common mistakes and fixes

Skipping email verification

What happens: Login will fail with a verification message.

Fix: Use the verify email link or resend verification endpoint.

Sharing reset links

What happens: Anyone with the link can reset within the valid window.

Fix: Treat reset links like private keys and use them once.

Expecting admins to recover passwords manually

What happens: Passwords are stored as hashes.

Fix: Use reset flow instead of asking for or setting known passwords outside authorized admin tools.

Security

Never ask users for passwords. Passwords are hashed with bcrypt. Sessions are stored in httpOnly cookies and public user JWTs require a strong `JWT_SECRET`.

Troubleshooting

Login rejected after registration

Likely cause: Email not verified.

First response: Use verify link or trigger resend verification.

Reset link expired

Likely cause: Reset token lasts one hour or was already used.

First response: Start forgot password again.

Notifications missing

Likely cause: No linked V-Vault response or wrong account.

First response: Confirm the user account used to submit the question.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Email verification is required before public login.

Public sessions are separate from admin sessions. | Session timeout and JWT handling in `src/lib/auth/session.ts`.

Admin Sign-In, Registration, Verification, Password Reset, Sessions, and Access Rules

Mental model

Admin auth is the locked staff entrance. Public users have a different door; admin sessions use a different cookie and stricter database revocation checks on APIs.

What it is

The admin identity layer for signing into the control plane, registering role-coded admins, verifying admin email, resetting passwords, changing passwords, and enforcing role access.

Who uses it

Internal staff with approved admin access; top-level admins manage admin accounts.

Where it lives

`/admin/sign-in`, `/admin/register`, `/admin/verify-email`, `/admin/forgot-password`, `/admin/reset-password`, `/api/admin/session`, `/api/admin/register`, `/api/admin/auth`.

What the public sees

The public site does not show admin tools. Admin routes are separate under `/admin`.

What admins control

Top-level admins manage admin accounts. Access codes map registration to moderator, editor, founder admin, or super admin.

Data saved

AdminUser records with role, email verification, password hash, failed login attempts, lockout, reset tokens, tokenVersion, active state, and audit logs.

Emails or notifications

Admin registration sends admin verification email. Verification can send welcome email. Forgot/reset sends admin password reset and password changed email.

Source truth

`src/lib/admin/session.ts; src/lib/admin/admin-auth.ts; src/lib/admin/api-guard.ts; src/lib/admin/page-guard.ts; admin auth routes and forms.`

How to operate it

1. Use `/admin/register` only for approved team members with the correct six-digit access code.
2. Verify the admin email before attempting normal sign-in.
3. Sign in at `/admin/sign-in`; the admin session lasts eight hours.
4. If sign-in fails after repeated attempts, wait for lockout expiry or have a top-level admin inspect the account.
5. Use `/admin/forgot-password` and `/admin/reset-password` for recovery. The current source does not include an active `/admin/recovery` route or `/api/admin/recovery/reactivate` route.

Common mistakes and fixes

Using public login for admin access

What happens: Public sessions do not unlock `/admin`.

Fix: Use `/admin/sign-in`.

Giving everyone a high-role access code

What happens: Too many people can edit sensitive content or view private data.

Fix: Issue the lowest role that can do the job.

Assuming page access equals API access

What happens: Admin pages check signed cookie; APIs also check database active/verified/tokenVersion.

Fix: If an API says unauthorized, verify the DB account state.

Security

Keep admin access codes and session secrets private. Rotate credentials if exposed. Do not put real access codes in public docs, screenshots, GitHub issues, or chat.

Troubleshooting

Admin sees sign-in loop

Likely cause: Cookie missing/invalid or role rejected.

First response: Clear cookies, sign in again, and inspect API auth result.

Email not verified error

Likely cause: Admin account exists but verify token not completed.

First response: Use `/admin/verify-email` or resend from verify flow.

Account inactive

Likely cause: A top-level admin disabled it or DB says inactive.

First response: Reactivate through Admin Accounts if appropriate.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Admin sessions use `dbfh_admin_session` ; public users use `user_session` .

Role rank is moderator < editor < founder_admin < super_admin. |
Exact cookie signing and role-code mapping in admin/session.ts.

Admin Dashboard, Platform Health, Site Settings, Site Alerts, and Audit Logs

Mental model

The dashboard is the control-room overview. Platform Health is the instrument panel. Settings are the public signboards. Alerts are temporary notices. Audit logs are the accountability trail.

What it is

The daily admin operating surface: dashboard metrics, platform health checks, site-wide settings, alert notices, and audit visibility.

Who uses it

Admins with relevant roles. Top-level admins own settings, account governance, and Platform Health. Editors and moderators work in lower-risk modules according to route/API guards.

Where it lives

/admin, /admin/health, /admin/settings, /admin/alerts, audit logs in repository and user detail surfaces.

What the public sees

Only the effects: updated hero/footer/contact data and home-page alerts.

What admins control

Dashboard shows counts. Platform Health reads dependency and route status. Site Settings updates brand/contact/hero/footer data. Alerts manage ticker notices. Audit logs record important actions.

Data saved

SiteSettings, SiteAlert, and AuditLog records. Platform Health itself is read-only; it queries existing tables and environment readiness.

Emails or notifications

Settings and alerts do not send email.

Source truth

src/app/admin/page.tsx; src/app/admin/health/page.tsx; src/lib/admin/health.ts; SiteSettingsForm.tsx; AlertsBoard.tsx; AuditLogViewer.tsx; repository audit methods.

How to operate it

1. Use Dashboard first to confirm the admin system can load and to see content counts.
2. Use Platform Health when any route, upload, email, database, or admin action fails and you need to identify the likely dependency.
3. Use Site Settings for rarely changed global text and contact links.
4. Use Site Alerts for time-bound public notices and remove or expire them when done.
5. Review audit logs when investigating who changed a sensitive record or account status.

Common mistakes and fixes

Using alerts as permanent content

What happens: Important information scrolls away and may be dismissed.

Fix: Put permanent content in normal pages or settings.

Editing settings during an incident without notes

What happens: The team loses context.

Fix: Record the reason in operating notes or issue tracker.

Ignoring audit failures in logs

What happens: Accountability becomes weaker.

Fix: Investigate repeated audit write errors.

Treating Platform Health as a full user test

What happens: A route may look healthy because dependencies are reachable, even if a copy/layout issue or browser-only bug still exists.

Fix: Use Platform Health to locate backend causes, then still smoke-test the affected public or admin workflow.

Security

Only top-level admins should change global site settings or view Platform Health. Alerts are public; never use them for private operations.

Troubleshooting

Dashboard counts are zero

Likely cause: Database not configured or unreachable.

First response: Open `/admin/health` and check database/model probes before making content decisions.

Settings save rejected

Likely cause: Role below top-level admin authority or validation failed.

First response: Use a top-level admin account and correct URL/contact fields.

Platform Health shows degraded

Likely cause: Optional service such as Resend or R2 is not configured, or a public route is using fallback because a dependency is unhealthy.

First response: Read the route row, dependency reason, operator action, and request/reference ID; then inspect the matching logs.

Audit history missing

Likely cause: No action logged or database issue.

First response: Check server logs and repository audit path.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Settings and Platform Health require top-level admin authority in practice. Alerts require moderator or higher in the current API guard.

Audit logs protect accountability but do not replace human operational notes. | Specific audit action names in repository methods and route definitions in admin/health.ts.

Media Library, R2 Uploads, and Asset Deletion Rules

Mental model

R2 is the warehouse; the database is the catalog. The site displays assets by URL, so deleting or changing files without updating the catalog breaks public content.

What it is

The admin media system for uploading images, documents, audio, and video to Cloudflare R2 and saving asset records in the database.

Who uses it

Editors upload; moderators view; top-level admins delete unused assets.

Where it lives

/admin/media, admin upload modal, /api/admin/media, /api/admin/media/presign, /api/admin/media/complete, /api/admin/media/[id].

What the public sees

Media appears wherever content references its public URL: hero images, team profiles, gallery, podcast audio/cover, article covers, and event covers.

What admins control

File upload, label, alt text, kind inferred from MIME type, asset listing, and delete for unused assets.

Data saved

MediaAsset records with label, storage key, bucket, URL, MIME type, size, kind, alt text, creator, and timestamps.

Emails or notifications

No email is sent.

Source truth

src/lib/admin/media-policy.ts; src/lib/storage/r2.ts; src/components/admin/media-upload.ts; MediaLibrary.tsx; AdminUploadModal.tsx; media API routes.

How to operate it

1. Choose the right file type and size: images up to 10MB, audio up to 80MB, video up to 200MB, documents up to 20MB, other files up to 5MB.
2. Upload through the admin module that needs the asset or through Media Library.
3. Use clear labels and alt text for images.
4. For large admin uploads, the browser requests a presigned R2 URL, uploads directly to R2, then completes the database record.

5. Before deleting, let the app check usage. Deletion is blocked if the asset is referenced by settings, team, gallery, podcast, or articles.

Common mistakes and fixes

Uploading unsupported files

What happens: Validation blocks the upload.

Fix: Use allowed MIME types: common images, audio, video, PDF/text/Word docs.

Deleting directly from R2

What happens: Database records can point to missing files.

Fix: Delete through admin when possible.

Bad R2 CORS settings

What happens: Browser presigned uploads fail.

Fix: Allow PUT from the admin domain with Content-Type header.

Security

Do not upload private records, identity documents, or confidential health files unless the storage policy and access model explicitly permit it.

Troubleshooting

Upload fails before progress

Likely cause: Validation or presign API error.

First response: Check file type, size, role, and R2 env variables.

Upload fails during transfer

Likely cause: Browser cannot PUT to R2 due to CORS/network.

First response: Fix R2 CORS and retry.

Delete says asset in use

Likely cause: Usage protection found references.

First response: Replace/remove those references, then delete.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Media deletion is protected when assets are still referenced.

Direct upload uses presigned URLs; direct server upload route also exists. | Exact MIME list and size limits in media-policy.ts.

Admin Content Boards: Team, Gallery, Events, Library, Podcast, and Reviews

Mental model

Each board is a publishing desk. Draft means not public, published means visible, archived means preserved but hidden from normal public lists.

What it is

The admin boards that turn staff content decisions into public records.

Who uses it

Editors and higher create/update most content; founder admins manage library; top-level admins delete sensitive records.

Where it lives

/admin/team, /admin/gallery, /admin/events, /admin/library, /admin/podcast, /admin/reviews.

What the public sees

Published content across the matching public pages.

What admins control

Each board controls fields, status, sort order, media references, and module-specific metadata.

Data saved

TeamMember, GalleryImage, OutreachEvent, Article, PodcastEpisode, Review, EventComment, and audit records.

Emails or notifications

Content publishing does not trigger email in the current code.

Source truth

Admin board components; admin API routes; Prisma content models; public page repositories.

How to operate it

1. Create new content as draft first.
2. Upload media through the board so the right URL is attached to the right field.
3. Fill required metadata, especially slugs, alt text, summaries, and status.
4. Preview the public route after publishing.
5. Archive content when it should disappear publicly but remain historically available.

Common mistakes and fixes

Publishing straight from incomplete drafts

What happens: Public pages look unfinished.

Fix: Use draft until content, image, and metadata are complete.

Deleting when archive is enough

What happens: The team loses history and audit context.

Fix: Archive first unless deletion is legally or operationally required.

Wrong role for a task

What happens: API returns insufficient permissions.

Fix: Check the RBAC table before assigning work.

Security

Use least privilege. Moderators can view and moderate; editors publish; founder admins manage library; top-level admins handle deletion and sensitive administration.

Troubleshooting

Board loads but save fails

Likely cause: Role too low, validation error, or database issue.

First response: Read the inline error and inspect admin API status.

Public page not updated

Likely cause: Status not published, cache/revalidate, or fallback data.

First response: Check record status and open the detail route directly.

Delete blocked

Likely cause: Role is not top-level admin or media/content is referenced.

First response: Use an authorized account and remove dependencies first.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Draft/published/archived is the core publishing model.

Different boards share media, audit, validation, and RBAC patterns. | Specific fields in each board component and Prisma model.

User Management, Admin Accounts, Activation, Deactivation, and RBAC

Mental model

Account management is the authority layer. It decides who can enter, what they can touch, and how access is revoked.

What it is

Internal control over public users and admin accounts, including role assignment, activation/deactivation, suspension, password setup, deletion, and account safety rules.

Who uses it

Top-level admins primarily. Lower admin roles should not manage accounts unless code changes.

Where it lives

/admin/users, /admin/users/[slug], /admin/admin-users, /api/admin/users/, /api/admin/admin-users/.

What the public sees

Public users only experience whether their own account is active, verified, and able to sign in.

What admins control

Public user activation/deactivation, user detail/audit view, admin account creation/update/suspend/delete, admin role, active state, phone, and temporary password.

Data saved

User, AdminUser, and AuditLog records, including failed login/lockout fields and tokenVersion for admin session invalidation.

Emails or notifications

Public identity flows send verification/reset/password emails where configured. Admin account create, edit, suspend, and delete flows attempt to email the affected admin; the UI tells the operator if the notification was not sent.

Source truth

AdminUsersBoard; UsersTable; AdminUserDetailClient; user/admin repository methods; RBAC/session modules.

How to operate it

1. Use User Management to inspect a public user, verify email/activity, and activate or deactivate when needed.
2. Use Admin Accounts only as a top-level admin.
3. Assign the lowest role that can complete the job.
4. Use Suspend for immediate reversible access removal. It sets the admin inactive and sends a suspension email when Resend is configured.
5. Use Delete only when removal is intentional and approved. The API blocks deleting or suspending the signed-in admin account.
6. When changing sensitive admin account state, expect tokenVersion changes to invalidate older sessions.
7. Check audit logs after account changes to confirm the action was recorded.

Common mistakes and fixes

Making too many top-level admins

What happens: Private data and destructive controls are overexposed.

Fix: Keep founder_admin and super_admin access small and intentional.

Deactivating the wrong user

What happens: A real user loses access.

Fix: Confirm email and user detail before action.

Removing the last super admin

What happens: The team can lock itself out.

Fix: Repository protections are designed to prevent losing the final active super admin.

Assuming account email was sent

What happens: The admin may not know their account was edited, suspended, or deleted if Resend is missing.

Fix: Read the success banner. If it says notification was not sent, check Platform Health and Resend configuration.

Security

RBAC rank is enforced in APIs. Page guards are lighter, but API guards verify signed cookie, database account, active state, email verification, and tokenVersion.

Troubleshooting

User cannot sign in

Likely cause: Inactive, email unverified, password issue, or lockout.

First response: Check user detail and guide them through verify/reset.

Admin API says unauthorized

Likely cause: Session revoked, inactive, unverified, or tokenVersion mismatch.

First response: Sign in again and inspect admin account state.

Role change not reflected

Likely cause: Old session or page cache.

First response: Sign out/in and check API access.

Suspend button disabled

Likely cause: You are viewing your own admin account or the account is already inactive.

First response: Use another top-level admin account for cross-admin governance, and never try to suspend yourself while signed in.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Top-level admin authority is required for user/admin account administration.

Suspend/delete are immediate access changes and should be communicated. | Exact role gate per route in api-guard.ts, page-guard.ts, account-notifications.ts, and admin-users route files.

Admin Module Operating Sheets

The chapters above explain the admin system as a whole. The following operating sheets are intentionally repetitive in a useful way: each one tells the operator what the module is, who should use it, what data it saves, what can go wrong, and how to troubleshoot it.

Admin Dashboard Operating Sheet

Mental model

The dashboard is the morning check. It tells you whether the platform looks alive and where attention is needed.

What it is

The dashboard summarizes key operational counts and gives the team direct paths into major admin modules.

Who uses it

All signed-in admins use it as their starting point.

Where it lives

/admin, src/app/admin/page.tsx, AdminDashboardCards.tsx, repository dashboard summary.

What the public sees

Nothing directly. The dashboard is internal.

What admins control

The dashboard itself is read-only. It links to settings, media, alerts, reviews, registration, and other modules.

Data saved

No dashboard data is saved by viewing. Counts are read from existing database tables.

Emails or notifications

No emails or notifications are triggered by dashboard viewing.

Source truth

`src/app/admin/page.tsx`; `src/components/admin/AdminDashboardCards.tsx`; `src/lib/admin/repository.ts`.

How to operate it

1. Sign in to `/admin/sign-in`.
2. Open `/admin` and check whether the dashboard loads without a data alert.
3. Scan counts for unusual changes: users, V-Vault submissions, reviews, events, alerts, and media assets.
4. Use the direct action links to enter the module that needs work.

Common mistakes and fixes

Ignoring a data load alert

What happens: Admin decisions may be based on incomplete counts.

Fix: Check database/API health before making operational decisions.

Treating counts as analytics

What happens: The dashboard counts records, not full product analytics.

Fix: Use analytics tooling for traffic and conversion questions.

Sharing dashboard screenshots publicly

What happens: Internal counts and module links may leak.

Fix: Keep admin screenshots internal.

Security

Dashboard access requires an admin session. Do not leave it open on shared devices.

Troubleshooting

Dashboard redirects to sign-in

Likely cause: Admin cookie missing or invalid.

First response: Sign in again.

Cards show zeros unexpectedly

Likely cause: Database not configured or unavailable.

First response: Check admin API and DB env.

A module link fails

Likely cause: Role may be insufficient.

First response: Check RBAC and use the right admin account.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Use dashboard first for a quick operational pulse.

Counts are database record counts, not marketing analytics. |
Dashboard summary query in the admin repository.

Platform Health Operating Sheet

Mental model

Platform Health is the operator's instrument panel. It does not press every button on the website for you, but it shows whether the main backend dependencies and data models are reachable before you start guessing.

What it is

A super-admin page and API snapshot that checks database configuration/reachability, Resend email configuration, Cloudflare R2 configuration, admin environment readiness, route groups, Prisma model probes, admin-account counts, and the error-reference table.

Who uses it

Top-level admins and technical maintainers during incidents, deployment checks, and confusing admin failures.

Where it lives

/admin/health, /api/admin/health, src/app/admin/health/page.tsx, src/app/api/admin/health/route.ts, and src/lib/admin/health.ts.

What the public sees

Nothing directly. This is an internal operations view.

What admins control

Platform Health is read-only. Admins refresh the snapshot and use its route/dependency rows to decide the next investigation step.

Data saved

No new platform data is saved by viewing the page. It queries existing tables such as AdminUser, User, SiteSettings, SiteAlert, MediaAsset, Article, OutreachEvent, EventComment, EventLike, ContactSubmission, VaultSubmission, VaultResponse, UserNotification, Review, TeamMember, GalleryImage, PodcastEpisode, AuditLog, and RateLimitBucket.

Emails or notifications

No email or notification is sent by viewing Platform Health.

Source truth

getAdminHealthSnapshot in `src/lib/admin/health.ts`; operator error codes in `src/lib/api/error-reference.ts`; structured server logging in `src/lib/api/observability.ts`.

How to operate it

1. Sign in as a top-level admin and open `/admin/health`.
2. Copy the Reference ID before investigating a serious failure.
3. Check the four summary cards: Database, API surface, Email, and Media storage.
4. If a route is blocked or degraded, read its Depends on, Cause, and Operator action columns.
5. Check Database Model Probes for the exact model that cannot be queried.

6. Use the Error Reference table to translate error codes such as `validation_failed`, `permission_denied`, `database_unavailable`, `storage_unavailable`, `service_unavailable`, and `server_error`.
7. Search server logs by the Reference ID when a 5xx or blocked probe appears.

Common mistakes and fixes

Expecting it to prove every UI works

What happens: Backend dependencies look healthy while a browser-only layout, copy, or interaction bug remains.

Fix: Use Platform Health for backend cause-finding, then still test the affected page or form.

Ignoring degraded email or storage

What happens: Sign-in may work, but reset emails, verification emails, uploads, or media publishing can fail.

Fix: Treat degraded services as real production work, even when public pages still load.

Sharing screenshots widely

What happens: Internal route names, counts, and dependency details may leak.

Fix: Keep screenshots inside the operations team and redact IDs where needed.

Security

Platform Health is top-level-admin-only. Do not publish its output, route map, model counts, request IDs, or provider status publicly.

Troubleshooting

Page redirects to sign-in

Likely cause: No valid admin session.

First response: Sign in at /admin/sign-in and confirm the account is active/verified.

Page says permission denied

Likely cause: Signed-in role is below top-level admin authority.

First response: Use an approved top-level admin account.

Database blocked

Likely cause: DATABASE_URL missing, CockroachDB unavailable, or migrations not applied.

First response: Check DATABASE_URL, run prisma migrate deploy in production, and compare the failing model probe with migrations.

Email degraded

Likely cause: RESEND_API_KEY or verified sender configuration is missing.

First response: Configure Resend before relying on verification, reset, or admin account notification emails.

Storage degraded

Likely cause: R2 account ID, bucket, access keys, public URL, or CORS policy is incomplete.

First response: Fix Cloudflare R2 configuration before troubleshooting media upload forms.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Platform Health tells you where to investigate first, not whether every visitor journey is perfect.

Degraded means some flows may still work while dependent flows are unsafe to trust. | Exact route definitions, dependency keys, and table probes in `src/lib/admin/health.ts`.

Site Settings Operating Sheet

Mental model

Site settings are not a content dumping ground. They are the small set of global facts the entire site leans on.

What it is

The global site settings editor for brand, hero, contact, and footer copy.

Who uses it

Top-level admins update it. Other admins may view settings through API paths but the page requires top-level authority.

Where it lives

/admin/settings, /api/admin/settings, SiteSettingsForm.tsx.

What the public sees

Updated site name, tagline, motto, contact email, WhatsApp link, hero headline/body/image/alt text, and footer blurb where used.

What admins control

All SiteSettings fields for the default scope.

Data saved

SiteSettings record with updatedBy relation and timestamps.

Emails or notifications

No email is sent.

Source truth

`src/app/admin/settings/page.tsx`; `src/app/api/admin/settings/route.ts`; `SiteSettingsForm.tsx`; `siteSettingsSchema`; `prisma SiteSettings`.

How to operate it

1. Sign in as a top-level admin and open `/admin/settings`.
2. Edit one logical group at a time: brand, contact, hero, or footer.
3. Upload hero media through the form if replacing the image.
4. Save and immediately open the public home page to confirm the result.
5. Keep a note of why major brand/contact changes were made.

Common mistakes and fixes

Putting temporary notices in hero copy

What happens: The public home page becomes stale after the notice expires.

Fix: Use Site Alerts for temporary notices.

Using an unapproved contact link

What happens: Visitors may contact the wrong channel.

Fix: Use official DownBelow contact channels only.

Changing siteUrl casually

What happens: Canonical links and emails may break.

Fix: Change base URL only as part of deployment/domain work.

Security

Do not paste private support notes, admin URLs, or secret links into settings fields.

Troubleshooting

Save rejected

Likely cause: Role below top-level admin authority or validation failed.

First response: Use a top-level admin account and correct invalid fields.

Public page unchanged

Likely cause: DB fallback/cache or save did not complete.

First response: Check API response and reload public page.

Hero image broken

Likely cause: Bad URL or media asset unavailable.

First response: Open the image URL directly and replace if needed.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Site Settings is a super-admin global control.

Changing siteUrl affects email links and SEO. | Exact field validation in siteSettingsSchema.

Site Alerts Operating Sheet

Mental model

An alert is a temporary public announcement, not a page, not a newsletter, and not an incident log.

What it is

The admin board for short, scheduled home-page notices.

Who uses it

Editors and higher manage alerts.

Where it lives

/admin/alerts, /api/admin/alerts, /api/alerts/active, SiteAlertTicker.tsx.

What the public sees

A dismissible Notice ticker on the home page when an active alert is within its schedule window.

What admins control

Alert text, speed, duration seconds, active flag, start time, and optional end time.

Data saved

SiteAlert records and audit logs for create/update/delete.

Emails or notifications

No email is sent.

Source truth

AlertsBoard.tsx; alert API routes; listPublicActiveSiteAlerts; SiteAlert model.

How to operate it

1. Create a short alert with one clear instruction.
2. Set start and end times when the notice is time-sensitive.
3. Preview the home page in a browser that has not dismissed the alert.
4. Deactivate or expire the alert when it is no longer useful.

Common mistakes and fixes

No end date for a deadline notice

What happens: Expired information can stay live.

Fix: Set an end date for campaign/event notices.

Too many active alerts

What happens: Ticker becomes noisy and less useful.

Fix: Keep active alerts to urgent current information.

Sensitive operational text

What happens: Private context becomes public.

Fix: Keep alerts public-safe.

Security

Alerts are public and browser-dismissible. Do not use alerts for private staff instructions.

Troubleshooting

Alert exists but does not show

Likely cause: Inactive, not started, expired, dismissed locally, or not on home page.

First response: Check schedule and test incognito on /.

Ticker speed feels wrong

Likely cause: Speed/duration settings are too high/low.

First response: Adjust within the board and preview.

API returns empty alerts

Likely cause: No currently active records.

First response: Check `isActive`, `startsAt`, and `endsAt`.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Only currently active alerts appear publicly.

Dismissal is stored locally in the visitor browser. | Alert timing logic in repository and ticker animation in component.

Team Management Operating Sheet

Mental model

Team profiles are public trust records. They need accuracy, consent, and consistent presentation.

What it is

The admin module for creating and maintaining public team profiles.

Who uses it

Editors create/update; top-level admins delete.

Where it lives

/admin/team, /api/admin/team, public /team and /about.

What the public sees

Published team member profiles grouped by tier.

What admins control

Name, slug, role, tier, credentials, image URL/upload, image alt, bio, status, and sort order.

Data saved

TeamMember records and audit logs.

Emails or notifications

No email is sent.

Source truth

TeamMembersBoard.tsx; team API routes; public team/about pages; TeamMember Prisma model.

How to operate it

1. Add the person as draft.

2. Upload an approved image and write alt text.
3. Choose the correct tier and sort order.
4. Write a bio that is useful but not private.
5. Publish and check public /team and /about.

Common mistakes and fixes

Publishing without consent

What happens: Privacy and trust risk.

Fix: Confirm the person approved profile content.

Weak image crop

What happens: Public card looks poor.

Fix: Use a clear portrait or approved team image.

Slug typo

What happens: Profile references become awkward or broken.

Fix: Use simple lowercase slugs before publishing.

Security

Do not publish private phone numbers, home addresses, internal notes, or unapproved credentials.

Troubleshooting

Profile missing

Likely cause: Status not published or tier filter issue.

First response: Check status and public route.

Delete unavailable

Likely cause: Role below top-level admin authority.

First response: Archive if deletion is not required.

Image fails

Likely cause: Bad R2 URL or asset removed.

First response: Replace image through upload flow.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Published team profiles are public trust signals.

Draft protects unfinished records from public display. | Required bio/image validations in TeamMembersBoard and schemas.

Gallery Management Operating Sheet

Mental model

Gallery images are public evidence. The metadata should tell the story without exposing private people.

What it is

The admin module for public gallery images and metadata.

Who uses it

Editors create/update; top-level admins delete.

Where it lives

/admin/gallery, /api/admin/gallery, public /gallery.

What the public sees

Published images in category filters and modal detail view.

What admins control

Title, slug, image, alt text, category, status, event name, location, captured date, sort order, caption, and description.

Data saved

GalleryImage records and audit logs.

Emails or notifications

No email is sent.

Source truth

GalleryImagesBoard.tsx; gallery API routes; ImageViewModal; GalleryImage model.

How to operate it

1. Upload the approved image.
2. Add alt text, title, caption, and description.
3. Select the right category and status.
4. Add event/location/date only when accurate.
5. Publish and check the modal on /gallery.

Common mistakes and fixes

Uploading images with private details

What happens: Participants or documents may be exposed.

Fix: Review image background and consent before publishing.

Thin description

What happens: The image loses context.

Fix: Explain what is happening and why it matters.

Wrong category

What happens: Visitors cannot find the image.

Fix: Use outreach/event/team/community/facility intentionally.

Security

Protect participant privacy. Be especially careful with minors, medical settings, and identifiable sensitive situations.

Troubleshooting

Image not in category

Likely cause: Wrong category or status.

First response: Edit category/status and reload.

Modal opens wrong image

Likely cause: Slug/query mismatch.

First response: Check slug uniqueness and URL.

Upload blocked

Likely cause: File type/size validation.

First response: Use allowed image types under 10MB.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Alt text and consent matter as much as image quality.

Gallery detail URLs use a modal query pattern. | Gallery category enum and field validation.

Events Management and Comment Moderation Operating Sheet

Mental model

An event record is both promotion and archive. It must be accurate before, during, and after the event.

What it is

The admin module for events and their public engagement controls.

Who uses it

Editors create/update events; moderators manage comments; top-level admins delete events.

Where it lives

/admin/events, event admin APIs, public /events and /events/[slug].

What the public sees

Published events, live indicators, stream embeds, likes, comments, and share controls.

What admins control

All event fields plus comment visibility status.

Data saved

OutreachEvent, EventComment, EventLike, and audit records.

Emails or notifications

No email is sent.

Source truth

EventsBoard.tsx; event admin/public API routes; events repository and schemas.

How to operate it

1. Create event as draft with stable slug and schedule.
2. Attach cover image and alt text.
3. Add stream URL/provider only when confirmed.
4. Publish and test the public page.
5. Moderate comments during and after the event.

Common mistakes and fixes

Wrong time zone/date

What happens: Visitors miss the event.

Fix: Confirm schedule in the intended local context before publishing.

Leaving engagement on during abuse

What happens: Comment spam may grow.

Fix: Hide/flag comments and consider disabling engagement.

Deleting event history

What happens: Public record disappears.

Fix: Archive unless deletion is required.

Security

Moderation should protect users from harassment, spam, and unsafe medical claims.

Troubleshooting

Comment status update fails

Likely cause: Role below moderator or invalid status.

First response: Use moderator+ and visible/hidden/flagged.

Event shows 404

Likely cause: Slug wrong or status not published.

First response: Check admin record.

Like count odd

Likely cause: Optimistic UI or unique user constraints.

First response: Reload and check DB if needed.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Engagement can be disabled per event.

Comment moderation is separate from deleting the whole event. |
Event schema and comment repository functions.

Library Article Management Operating Sheet

Mental model

Articles are long-lived educational assets. They should be medically responsible, searchable, and easy to revisit.

What it is

The editorial module for health library articles.

Who uses it

Founder admin and super admin both have top-level authority for library articles in the current RBAC model.

Where it lives

/admin/library, library API routes, public /library.

What the public sees

Published articles in search/category listing and article detail pages.

What admins control

Title, slug, summary, body, topic/category, author, cover image, status, publish date, sort order.

Data saved

Article records and audit logs.

Emails or notifications

No email is sent.

Source truth

LibraryArticlesBoard.tsx; library API routes; library repository; Article model.

How to operate it

1. Draft the article with a clear educational goal.
2. Choose category and slug before publishing.
3. Add summary and cover image.
4. Review for medical safety and tone.
5. Publish and verify related articles and disclaimer display.

Common mistakes and fixes

Using vague titles

What happens: Users cannot scan or search effectively.

Fix: Use direct educational titles.

No qualified review for sensitive advice

What happens: Users may misunderstand health guidance.

Fix: Review before publishing.

Publishing duplicate categories/slugs carelessly

What happens: Library organization becomes messy.

Fix: Use consistent categories.

Security

Avoid personalized medical instructions in public articles. Keep guidance educational and appropriately qualified.

Troubleshooting

Article not in sitemap

Likely cause: Not published or DB read unavailable.

First response: Check publication status and sitemap generation.

Cover missing

Likely cause: Asset URL broken.

First response: Replace with a valid media URL.

Save permission denied

Likely cause: Role below founder_admin.

First response: Use authorized role.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Library is founder-admin-level editorial content.

Public fallback articles may appear if DB is down. | Article validation and fallback seed behavior.

Podcast Management Operating Sheet

Mental model

The podcast board connects a large audio file to searchable public context.

What it is

The admin module for audio episodes and their supporting metadata.

Who uses it

Editors create/update; top-level admins delete.

Where it lives

/admin/podcast, podcast API routes, public /podcast.

What the public sees

Published audio episodes with player, details, transcript, show notes, and related episodes.

What admins control

Audio upload/URL, cover image, title, slug, summary, show notes, transcript, guest, tags, external source, status, publish date, sort order.

Data saved

PodcastEpisode records and media assets.

Emails or notifications

No email is sent.

Source truth

PodcastEpisodesBoard.tsx; podcast API routes; PodcastEpisode model; media upload code.

How to operate it

1. Prepare final audio before upload.
2. Upload audio and optional cover art.
3. Enter tags, guest, transcript, and show notes.
4. Publish and test audio controls on desktop and mobile.
5. Archive old episodes only when they should stop showing publicly.

Common mistakes and fixes

Uploading drafts as final

What happens: Public listeners hear unfinished content.

Fix: Only upload approved audio.

Missing transcript

What happens: Accessibility and search are weaker.

Fix: Add transcript for important episodes.

Huge unsupported file

What happens: Upload fails.

Fix: Use supported formats and size limits.

Security

Confirm consent for guest names and sensitive stories before publishing.

Troubleshooting

Audio player blank

Likely cause: Bad URL or unsupported MIME type.

First response: Open URL directly and check MIME.

Upload progress stalls

Likely cause: R2 CORS/network issue.

First response: Check R2 bucket CORS.

Episode absent publicly

Likely cause: Draft/archived or publish date issue.

First response: Check status.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Podcast audio is a media asset plus a database episode record.

Transcript is operationally valuable, not decorative. |

PodcastEpisode fields and upload helper.

Review Moderation and Team Replies Operating Sheet

Mental model

Reviews are public proof, but proof must be curated responsibly.
Consent and privacy come first.

What it is

The admin module for managing public testimonials and official team replies.

Who uses it

Moderators view; editors create/update; top-level admins delete.

Where it lives

/admin/reviews, review admin APIs, public /review.

What the public sees

Published reviews, ratings, helpful marks, and team replies.

What admins control

Review status, rating, source, author display fields, body, location, published date, sort order, admin reply.

Data saved

Review, ReviewHelpful, and audit records.

Emails or notifications

No email is sent.

Source truth

ReviewsBoard.tsx; reviews repository; review API routes; Review model.

How to operate it

1. Check new public submissions regularly because they are currently published immediately.
2. Archive or edit content that exposes private health details.
3. Add a team reply when it adds clarity or warmth.
4. Keep ratings and body truthful; do not manufacture testimonials.
5. Delete only when necessary and authorized.

Common mistakes and fixes

Over-editing user voice

What happens: Review may feel fake.

Fix: Only correct privacy, clarity, or safety issues.

Replying with medical diagnosis

What happens: Team reply becomes risky advice.

Fix: Keep replies supportive and direct people to proper care.

Ignoring public submissions

What happens: Unsafe content may stay public.

Fix: Make review checks part of daily operations.

Security

Team replies are official public communication. Keep them professional and non-diagnostic.

Troubleshooting

Review status update fails

Likely cause: Role below editor or validation error.

First response: Use editor+ and valid status.

Helpful count confusing

Likely cause: Anonymous visitor cookie or fallback seed review.

First response: Treat as lightweight signal.

Public page shows seed reviews

Likely cause: DB unavailable or empty published list.

First response: Check DB and published records.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Public review submission publishes immediately in current code.

Admin replies appear publicly. | Review source/status behavior in reviews repository.

User Management Operating Sheet

Mental model

User management is support plus safety. It should help real users recover access while protecting the platform from abuse.

What it is

The top-level-admin module for inspecting and managing public user accounts.

Who uses it

Top-level admins.

Where it lives

/admin/users, /admin/users/[slug], user admin APIs.

What the public sees

Only account effects: active/inactive state, email verification state, and ability to use authenticated features.

What admins control

Search/filter users, inspect profile/security fields, activate/deactivate, view audit logs and V-Vault submission count.

Data saved

User and AuditLog records.

Emails or notifications

Account lifecycle templates exist and may be sent by relevant repository calls where wired.

Source truth

UsersTable.tsx; AdminUserDetailClient.tsx; user admin API routes; user-repository.ts.

How to operate it

1. Search by email/name when helping a user.
2. Confirm identity through approved support process before discussing account details.
3. Check email verified, active state, lockout, and last activity.
4. Use activate/deactivate intentionally and record context outside the app if policy requires.
5. Send users through public reset flows instead of handling passwords directly.

Common mistakes and fixes

Deactivating by name only

What happens: Wrong person may lose access.

Fix: Confirm email and record details.

Discussing private V-Vault content in general support channels

What happens: Sensitive data leaks.

Fix: Escalate securely to the top-level admin workflow.

Trying to read passwords

What happens: Impossible by design.

Fix: Use reset flow.

Security

Public user details are private operational data. Limit screenshots and exports.

Troubleshooting

User locked out

Likely cause: Failed login attempts.

First response: Wait or reset according to policy.

User says verified but login fails

Likely cause: Wrong account or stale token.

First response: Check account emailVerified and active state.

Audit missing

Likely cause: Action may not log or DB write failed.

First response: Check server logs and repository flow.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Never ask for or expose user passwords.

Email verification and active state are separate. | User lockout and session logic in auth/user repository files.

Admin Account Management Operating Sheet

Mental model

Admin accounts are keys to the building. The fewer master keys, the safer the building.

What it is

The top-level-admin module for internal admin accounts, roles, active state, suspension, deletion, and temporary password setup.

Who uses it

Top-level admins only.

Where it lives

/admin/admin-users, admin-users API routes, AdminAccountsBoard.

What the public sees

Nothing directly.

What admins control

Admin name, email, phone, role, active flag, password setup/update, suspend, and delete.

Data saved

AdminUser records and audit logs.

Emails or notifications

Admin verification, welcome, password changed, account updated, account suspended, and account deleted templates are available. Create/edit/suspend/delete account flows attempt to notify the affected admin and report if email was not sent.

Source truth

AdminAccountsBoard.tsx; admin-users API routes including /api/admin/admin-users/[id]/suspend; account-notifications.ts; admin repository; AdminUser model.

How to operate it

1. Create admin accounts only for approved staff.
2. Assign the lowest role needed.
3. Require email verification for sign-in flows where applicable.
4. Suspend accounts immediately when access must stop but deletion is not yet appropriate.
5. Delete only when removal is approved and no longer needed for accountability.
6. Review active top-level access before risky account changes, and make sure at least one active literal super_admin remains.
7. Read the banner after edit/suspend/delete to confirm whether the notification email was sent.

Common mistakes and fixes

Sharing admin accounts

What happens: Audit trail loses meaning.

Fix: Each admin needs their own account.

Too many top-level admins

What happens: Private data and destructive actions are overexposed.

Fix: Keep founder_admin and super_admin accounts rare.

Deleting instead of deactivating during uncertainty

What happens: History and recovery become harder.

Fix: Suspend first unless deletion is required.

Ignoring the email status banner

What happens: The operator assumes the affected admin was notified even when the email provider is not configured.

Fix: If the banner says notification was not sent, check Resend and follow internal communication policy.

Security

Protect access codes and admin session secrets. Rotate if exposed.

Troubleshooting

New admin cannot sign in

Likely cause: Email unverified, inactive, wrong password, or role issue.

First response: Check account and verify flow.

Role does not unlock page

Likely cause: Session stale or page/API requires higher role.

First response: Sign out/in and confirm RBAC.

Cannot remove super admin

Likely cause: Last-super-admin protection.

First response: Create/verify another super admin first if intentional.

Cannot suspend admin

Likely cause: The target account is your own signed-in account, already inactive, or your role is below top-level admin authority.

First response: Use a different top-level admin account and confirm target status.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Top-level admin access is sensitive and should be rare.

Suspension and deletion remove access immediately and attempt an email notice. | Admin account safety rules in repository and account notification templates.

V-Vault Moderation Operating Sheet

Mental model

V-Vault moderation is a sensitive clinical-adjacent workflow. Privacy, tone, and auditability matter more than speed.

What it is

The admin workflow for reviewing sensitive questions, writing private replies, revealing identity only when authorized, and managing FAQ-eligible content.

Who uses it

Moderators can view masked submissions. Editors and higher can update moderation and send private responses. Top-level admins can reveal identity.

Where it lives

/admin/vault, /api/admin/vault, /api/admin/vault/[id]/respond.

What the public sees

Private replies in /me and approved FAQ-style preview content where applicable.

What admins control

Submission status, moderation notes, FAQ title, identity reveal, private response.

Data saved

VaultSubmission, VaultResponse, VaultSubmissionEvent, UserNotification, AuditLog.

Emails or notifications

Private response creates in-app notification; no response email is sent by current repository code.

Source truth

VaultModerationBoard.tsx; admin vault routes; admin repository vault methods; Vault models.

How to operate it

1. Open Admin V-Vault with at least moderator access.
2. Read the question without revealing identity unless you are a top-level admin and there is a real operational reason.
3. If you are editor or higher, set status and moderation notes.
4. If you are editor or higher, write a private response that is clear, safe, and not overreaching medically.
5. Submit response and confirm the user notification was created.

Common mistakes and fixes

Revealing identity casually

What happens: Privacy risk and audit trail sensitivity.

Fix: Reveal only when operationally necessary.

Turning private answers into FAQ without sanitizing

What happens: Private details may become public.

Fix: Only approve generalized, safe FAQ material.

Assuming email went out

What happens: User may only have in-app notification.

Fix: Tell users to check /me.

Security

Identity reveal is top-level-admin-only and audit-logged. Treat screenshots and exports as sensitive.

Troubleshooting

Cannot open admin vault

Likely cause: Role below moderator.

First response: Use an authorized admin account and confirm the required role for the specific action.

Response save fails

Likely cause: Validation, DB issue, or session problem.

First response: Check response body, API status, and DB.

User notification absent

Likely cause: Submission may not be linked to user or transaction failed.

First response: Inspect submission userId and repository result.

Knowledge split

Must know by heart

Must recognize | Lookup-only

V-Vault viewing, response, and identity reveal do not have the same role threshold.

FAQ approval is different from private response. | Status enum, canModerateVault, canViewVaultIdentity, and event creation logic in Prisma/repository.

Audit Logs Operating Sheet

Mental model

Audit logs answer: who touched what, when, and why the system thinks it happened.

What it is

The accountability layer that records important admin, user, and system actions.

Who uses it

Top-level admins and maintainers use it for investigations; all operators benefit from its discipline.

Where it lives

AuditLog model, repository writeAuditLog, user detail audit viewer, auth/account/content repository calls.

What the public sees

Nothing directly.

What admins control

Audit logs are generally written by the system rather than manually edited.

Data saved

Action, entity type/id, actor email/role, summary, metadata, IP address, user agent, success flag, timestamps.

Emails or notifications

No email is sent by audit logging.

Source truth

prisma AuditLog model; repository audit methods;
AuditLogViewer.tsx; auth/admin/user repository calls.

How to operate it

1. Use audit logs when investigating account access, moderation, deletion, or sensitive V-Vault actions.
2. Read the action and entity fields first.
3. Compare actor email/role with the expected operator.
4. Use metadata for context, but avoid treating it as a complete incident report.
5. Escalate repeated failed or missing audit writes to engineering.

Common mistakes and fixes

Treating audit logs as conversation notes

What happens: They lack human context.

Fix: Use an incident log or ticket for decisions and rationale.

Ignoring actor role

What happens: A role mismatch may reveal access issues.

Fix: Check whether the role should have had permission.

Exporting logs casually

What happens: Private emails/IP/user agents can leak.

Fix: Limit exports to authorized investigations.

Security

Audit data can include sensitive metadata. Share only with people who need it.

Troubleshooting

Expected audit missing

Likely cause: That path may not log or DB write failed.

First response: Inspect repository method and logs.

Unknown actor

Likely cause: Legacy/system action or fallback actor.

First response: Trace action source.

Many failures

Likely cause: Auth abuse or service issue.

First response: Investigate immediately.

Knowledge split

Must know by heart

Must recognize | Lookup-only

Audit logs support accountability but do not replace policy.

Some audit writes are non-blocking by design. | Exact audit action strings in repository/auth code.

Part IV - Backend and Production Operations

Chapter 1 - Next.js App Router Structure

The app uses Next.js App Router. Pages live under `src/app`. Public pages and admin pages are route folders. API endpoints are `route.ts` files under `src/app/api`. Shared business logic lives in `src/lib`. Reusable UI lives in `src/components`.

Public pages

Main path: `src/app/*/page.tsx`

Operator meaning: What visitors see.

Admin pages

Main path: `src/app/admin/*/page.tsx`

Operator meaning: What internal staff use.

Public APIs

Main path: `src/app/api/*/route.ts`

Operator meaning: Data actions from public forms and widgets.

Admin APIs

Main path: `src/app/api/admin/*/route.ts`

Operator meaning: Protected admin actions.

Business logic

Main path: `src/lib/*`

Operator meaning: Validation, sessions, repositories, email, storage, database.

Database schema

Main path: `prisma/schema.prisma`

Operator meaning: The data model contract.

Tip

When debugging, find the route first, then the component, then the repository method, then the Prisma model. That path keeps you from guessing.

Chapter 2 - Prisma and CockroachDB Data Model

Prisma is the typed database layer. CockroachDB speaks the PostgreSQL protocol and stores the operational records. The schema is the agreement between code and database. If the schema says a model has a field, the app expects the database table to match.

AdminUser

Admin identities, roles, verification, reset tokens, logout, tokenVersion.

User

Public users, verification, reset tokens, activity, notifications, Vault links.

SiteSettings

Global public site settings.

SiteAlert

Home-page active alert ticker records.

MediaAsset

R2 file catalog entries.

AuditLog

Accountability records for admin/user/system actions.

VaultSubmission/VaultResponse/VaultSubmissionEvent

Private V-Vault question and response workflow.

UserNotification

In-app notifications, especially V-Vault response alerts.

Article

Health library articles.

OutreachEvent/EventLike/EventComment

Events, engagement, and moderation.

ContactSubmission

Contact form records.

Review/ReviewHelpful

Public testimonials and helpful marks.

TeamMember

Published team profiles.

GalleryImage

Public gallery images.

PodcastEpisode

Audio episode metadata.

Chapter 3 - Migrations

A migration is a controlled database change. Use migrations so the database evolves in step with Prisma schema changes.

pnpm db:migrate

Use it when: Developing locally and intentionally creating a new migration from schema changes.

Do not use it when: Running production deploys. It may prompt and expects development-style workflows.

pnpm db:deploy

Use it when: Applying existing migrations to staging or production.

Do not use it when: You have not reviewed the migration files.

pnpm db:generate

Use it when: After schema changes or install to refresh Prisma Client.

Do not use it when: As a replacement for migrations. It does not change the database.

pnpm db:studio

Use it when: Inspecting data locally/admin with care.

Do not use it when: Editing production data casually.

Warning

Production uses `migrate deploy`, not `migrate dev`. In the current `package.json`, `pnpm build` runs `prisma generate` & next build. Apply production migrations separately with `pnpm db:deploy` or `pnpm release:migrate` before the production build/deploy process.

Chapter 4 - Cloudflare R2 Media Storage

Cloudflare R2 stores files; the database stores references to those files. The app can upload directly through the server or use a presigned URL so the browser sends the file to R2.

image

Allowed size: 10MB

Examples: JPEG, PNG, WebP, GIF, AVIF

audio

Allowed size: 80MB

Examples: MP3, MP4 audio, WebM, OGG, WAV

video

Allowed size: 200MB

Examples: MP4, WebM

document

Allowed size: 20MB

Examples: PDF, text, Word documents

other

Allowed size: 5MB

Examples: Fallback for supported but uncategorized files

Security

R2 credentials belong in environment variables only. Do not paste account IDs, access keys, or secret keys into documentation screenshots.

Chapter 5 - Resend Email System

Resend sends transactional emails for identity and account flows. The send function intentionally returns a skipped result instead of crashing when RESEND_API_KEY is missing, which is useful in development but dangerous if unnoticed in production.

Template

When used

verifyEmail

Public user registration and resend verification.

passwordReset

Public forgot password.

welcomeUser

Available template for public welcome messaging.

passwordChanged

Public/admin password reset or change flows.

accountDeactivated/accountDeleted

Available account lifecycle templates.

**adminAccountCreated/adminAccountUpdated/
adminAccountSuspended/adminAccountDeleted**

Admin account governance emails sent or attempted when top-level admins create, edit, suspend, or delete an admin account.

vaultResponseReady

Template exists, but current Vault response path creates in-app notification only.

verifyAdminEmail

Admin registration/resend verification.

welcomeAdmin

Admin email verification success.

adminPasswordReset

Admin forgot password.

adminAccountReactivated

Available for account reactivation messaging where wired.

Chapter 6 - Environment Variables

NEXT_PUBLIC_SITE_URL

Purpose: Base URL for links, metadata, sitemap, emails.

Safety note: Public value, but keep it correct for production.

DATABASE_URL / DIRECT_URL

Purpose: CockroachDB connection strings.

Safety note: Secret. Never publish.

JWT_SECRET

Purpose: Signs public user sessions.

Safety note: Secret, at least 32 chars, not a placeholder.

ADMIN_SESSION_SECRET

Purpose: Signs admin cookies.

Safety note: Secret, at least 32 chars, not a placeholder.

ADMIN_ACCESS_CODE

Purpose: Moderator registration code.

Safety note: Secret six-digit code.

ADMIN_SUPER_ADMIN_ACCESS_CODE

Purpose: Super admin registration code.

Safety note: Highly sensitive six-digit code.

ADMIN_FOUNDER_ADMIN_ACCESS_CODE

Purpose: Founder admin registration code.

Safety note: Secret six-digit code.

ADMIN_EDITOR_ACCESS_CODE

Purpose: Editor registration code.

Safety note: Secret six-digit code.

ADMIN_ALLOWED_USERS

Purpose: Seed/allowed admin list used by seed tooling.

Safety note: Treat as internal.

ADMIN_INVITE_TOKENS

Purpose: Optional invite-token allowlist for admin registration.

Safety note: Treat invite tokens like secrets. Do not place real tokens in screenshots or docs.

R2_ACCOUNT_ID / R2_ACCESS_KEY_ID / R2_SECRET_ACCESS_KEY / R2_BUCKET / R2_PUBLIC_URL

Purpose: Cloudflare R2 storage.

Safety note: Secret except public asset URL.

RESEND_API_KEY / RESEND_FROM_EMAIL / RESEND_FROM_NAME

Purpose: Transactional email provider and sender.

Safety note: API key secret; sender domain must be verified.

VAULT_SUBMISSIONS_ENABLED

Purpose: Turns public V-Vault submission on/off.

Safety note: Use false during pauses/incidents.

Chapter 7 - Rate Limiting, Fallback Behavior, and Security Rules

The app uses `checkRateLimit` for contact, V-Vault, public auth, review submission, admin auth, and admin registration flows. When `DATABASE_URL` is configured outside tests, limits are stored in the `RateLimitBucket` table. If the database-backed limiter fails, the code logs the failure and falls back to an in-memory limiter so the request path still has some protection.

This matters operationally: database-backed limits are shared across server instances, while the memory fallback is per process. If Platform Health shows the database blocked, do not treat rate limiting as fully durable until the database is healthy again.

Contact form

5 requests per IP per 10 minutes.

V-Vault submission

10 requests per IP per 10 minutes.

Public login

5 per 15 minutes.

Public registration

3 per hour.

Public forgot/reset/resend patterns

Separate IP and email limits in route handlers.

Admin login

IP and email limits plus account lockout.

Public fallback behavior matters. Some public pages use `readPublicDatabase` with a two-second timeout, 60-second cache, and static fallback. Admin writes do not silently fall back; they should fail clearly with service errors when the database or storage is not ready.

Security

Public fallback is for availability, not editing. If public content looks old during a database problem, do not assume the data disappeared. Check database health first.

Chapter 8 - Deployment and Production Checklist

Secrets

All production secrets are real, strong, and stored in the deployment provider only.

Database

DATABASE_URL works and migrations have been applied with `pnpm db:deploy`.

Prisma

`pnpm db:generate` completed and build can import Prisma Client.

Email

Resend API key and verified sender domain work; test verify/reset emails.

R2

Bucket, public URL, access keys, and CORS support uploads from the admin domain.

Admin bootstrap

At least one active verified `super_admin` exists, and top-level admin access has been reviewed.

Vault

VAULT_SUBMISSIONS_ENABLED intentionally true or false.

SEO

Correct NEXT_PUBLIC_SITE_URL, robots, sitemap, manifest, and metadata.

Tests

pnpm verify:release passes before release.

Smoke test

Public home, library, events, auth, /me, admin sign-in, media upload, and one save per critical board work.

Part V - Operational Checklists and Troubleshooting

Daily Operating Checklist

Check admin dashboard

Owner: Admin operator

Done when: Database-ready state and counts look reasonable.

Review new V-Vault submissions

Owner: Moderator for masked review, editor+ for response/moderation, top-level admin for identity reveal.

Done when: Sensitive questions have status and response plan.

Review public reviews

Owner: Moderator/editor

Done when: Unsafe or private content is archived or edited.

Check event comments

Owner: Moderator

Done when: Inappropriate comments hidden/flagged.

Check public home page

Owner: Operator

Done when: Hero, alert, and navigation work.

Check email incident reports

Owner: Operator/developer

Done when: Verification/reset messages are not failing silently.

Publishing Checklist

Before publishing content

Why it matters

Confirm status is draft until review is complete.

Prevents unfinished public pages.

Use stable slug.

Prevents broken shared links.

Add alt text for every meaningful image.

Accessibility and SEO.

Check medical/sensitive claims.

Protects users and DownBelow.

Open the public page after save.

Catches broken images, bad dates, or missing sections.

Archive instead of delete when history matters.

Preserves context and audit trail.

Troubleshooting Guide

Public page shows old fallback content

Likely cause: Database unavailable or fallback cache active.

First response: Open `/admin/health`, check database/model probes, then inspect recent logs by Reference ID if a server error occurred.

Escalate when: Admin writes also fail or DB is down.

Admin cannot save

Likely cause: Role too low, validation failed, or service unavailable.

First response: Read the API error, verify role, and use `/admin/health` to check database, email, storage, and route dependency status.

Escalate when: 503 or repeated DB/R2 errors.

Verification email not received

Likely cause: Resend missing, sender not verified, spam, or rate limit.

First response: Check Resend config in /admin/health and ask the user to search inbox/spam.

Escalate when: Provider shows failed delivery.

Password reset token invalid

Likely cause: Expired, used, or wrong token.

First response: Start forgot password again.

Escalate when: Repeated valid tokens fail.

Media upload fails

Likely cause: File validation, R2 env, CORS, or network.

First response: Check type/size, then check Platform Health for storage configuration before changing content records.

Escalate when: Direct R2 PUT fails repeatedly.

V-Vault response not visible

Likely cause: Wrong account, no response, or notification unread issue.

First response: Check user thread and notification record.

Escalate when: Records exist but /me cannot load them.

Admin locked out

Likely cause: Failed login attempts or inactive/unverified account.

First response: Wait lockout or use top-level admin account management.

Escalate when: No active super admin exists.

Admin account notification not sent

Likely cause: Resend API key/sender missing or provider delivery failed.

First response: Read the admin account banner, check Email on / admin/health, and verify Resend configuration before retrying communication.

Escalate when: Email is configured but provider logs show repeated failures.

Event comments spammed

Likely cause: Abuse within engagement feature.

First response: Hide/flag comments and consider disabling engagement.

Escalate when: Abuse continues across users/IPs.

Glossary

API route

A server endpoint that receives a form/action request and returns data.

App Router

Next.js folder-based routing system under `src/app`.

Audit log

A record of who did what, when, and to which entity.

Cookie

Small browser-stored token used here for sessions.

Draft

Saved but not intended for public display.

Published

Visible to the public where the page displays it.

Archived

Hidden from normal public display but preserved.

Prisma

The database toolkit that maps TypeScript code to database records.

R2

Cloudflare object storage for media files.

RBAC

Role-based access control.

Slug

The URL-friendly identifier for a record, such as an article or event.

V-Vault

Private question submission and response workflow.

Appendix A - Commands

pnpm install

Install dependencies.

pnpm dev

Run the local development server.

pnpm build

Generate Prisma Client and run the Next.js production build. It does not apply migrations.

pnpm start

Start the production server after build.

pnpm lint

Run linting.

pnpm test

Run Jest tests.

pnpm verify:release

Run TypeScript check, lint, unit tests, and integration tests.

pnpm db:generate

Generate Prisma Client.

pnpm db:migrate

Create/apply development migrations.

pnpm db:deploy

Apply existing migrations in staging/production.

pnpm db:seed

Seed database content.

pnpm db:studio

Open Prisma Studio.

Appendix B - Feature Coverage Checklist

Public page

Route/module: /

Coverage: Home, site settings, V-Vault preview, latest articles

Status: Covered

Public page

Route/module: /home

Coverage: Redirects to /

Status: Covered

Public page

Route/module: /about

Coverage: Mission, vision, team preview

Status: Covered

Public page

Route/module: /contact

Coverage: Contact request form

Status: Covered

Public page

Route/module: /events and /events/[slug]

Coverage: Events, streams, likes, comments

Status: Covered

Public page

Route/module: /library and /library/[slug]

Coverage: Articles, categories, detail pages

Status: Covered

Public page

Route/module: /outreach

Coverage: Static outreach content

Status: Covered

Public page

Route/module: /podcast and /podcast/[slug]

Coverage: Episode listing/detail/audio

Status: Covered

Public page

Route/module: /gallery and /gallery/[slug]

Coverage: Gallery grid/modal/detail redirect

Status: Covered

Public page

Route/module: /team

Coverage: Team directory

Status: Covered

Public page

Route/module: /review

Coverage: Reviews, public submission, helpful marks

Status: Covered

Public page

Route/module: /vault

Coverage: Authenticated V-Vault submission

Status: Covered

Public page

Route/module: /privacy and /terms

Coverage: Legal information routes

Status: Covered

Public auth

Route/module: /register /login /forgot-password /reset-password /verify-email

Coverage: Public identity lifecycle

Status: Covered

Public page

Route/module: /me

Coverage: Profile, password change, notifications

Status: Covered

SEO/PWA

Route/module: robots sitemap manifest metadata structured data

Coverage: Search/install support

Status: Covered

Public API

Route/module: /api/auth/register login logout session forgot-password reset-password verify-email resend-verification

Coverage: Public authentication actions

Status: Covered

Public API

Route/module: /api/contact

Coverage: Contact submissions and contact rate limit

Status: Covered

Public API

Route/module: /api/events /api/events/[slug] /comments /like

Coverage: Published events, likes, comments

Status: Covered

Public API

Route/module: /api/gallery /api/gallery/[slug]

Coverage: Gallery records and detail lookup

Status: Covered

Public API

Route/module: /api/reviews /api/reviews/[id]/helpful

Coverage: Review submission and helpful marks

Status: Covered

Public API

Route/module: /api/users/me /api/users/notifications /read

Coverage: Profile and notifications

Status: Covered

Public API

Route/module: /api/vault /api/vault/me

Coverage: V-Vault submit and thread history

Status: Covered

Public API

Route/module: /api/alerts/active

Coverage: Active site alert ticker feed

Status: Covered

Admin page

Route/module: /admin

Coverage: Dashboard

Status: Covered

Admin page

Route/module: /admin/health

Coverage: Top-level-admin Platform Health dashboard, dependency status, route map, model probes, and operator error reference

Status: Covered

Admin auth

Route/module: /admin/sign-in /register /verify-email /forgot-password /reset-password

Coverage: Admin identity lifecycle

Status: Covered

Admin requested feature

Route/module: /admin/recovery

Coverage: Not active in current route inventory; current flow uses verify/reset/account activation

Status: Documented as not active

Admin page

Route/module: /admin/settings

Coverage: Site settings

Status: Covered

Admin page

Route/module: /admin/alerts

Coverage: Site alert ticker controls

Status: Covered

Admin page

Route/module: /admin/media

Coverage: Media library and R2 upload

Status: Covered

Admin page

Route/module: /admin/team

Coverage: Team management

Status: Covered

Admin page

Route/module: /admin/gallery

Coverage: Gallery management

Status: Covered

Admin page

Route/module: /admin/events

Coverage: Events and comment moderation

Status: Covered

Admin page

Route/module: /admin/library

Coverage: Article management

Status: Covered

Admin page

Route/module: /admin/podcast

Coverage: Podcast management

Status: Covered

Admin page

Route/module: /admin/reviews

Coverage: Review moderation and replies

Status: Covered

Admin page

Route/module: /admin/users and /admin/users/[slug]

Coverage: Public user management and audit detail

Status: Covered

Admin page

Route/module: /admin/admin-users

Coverage: Admin account management, edit/delete/suspend, notification email status

Status: Covered

Admin page

Route/module: /admin/vault

Coverage: V-Vault moderation/private response

Status: Covered

Admin API

Route/module: /api/admin/session register verify-email forgot-password reset-password change-password

Coverage: Admin authentication and account recovery

Status: Covered

Admin API

Route/module: /api/admin/health

Coverage: Top-level-admin health snapshot for database, email, storage, admin env, routes, table probes, and error reference

Status: Covered

Admin API

Route/module: /api/admin/settings alerts media team gallery events library podcast reviews users admin-users vault

Coverage: Admin control plane actions, RBAC, admin account edit/delete, and V-Vault moderation

Status: Covered

Admin API

Route/module: /api/admin/admin-users/[id]/suspend

Coverage: Top-level-admin suspension flow with self-suspend protection and attempted email notification

Status: Covered

Admin API

Route/module: /api/admin/media/presign /complete /[id]

Coverage: R2 upload and protected deletion flow

Status: Covered

Admin API

Route/module: /api/admin/events/[id]/comments/[commentId]

Coverage: Event comment moderation

Status: Covered

Backend

Route/module: src/lib/env.ts prisma.ts public-database.ts storage/r2.ts email/* rate-limit.ts

Coverage: Operations and infrastructure

Status: Covered

Appendix C - Roles, Environment, and Data Model Quick Tables

Roles

moderator

Rank: 1

Use for: Viewing/moderating allowed areas.

Avoid using for: Publishing content or account management.

editor

Rank: 2

Use for: Publishing most content, alerts, uploads.

Avoid using for: V-Vault identity, admin accounts, destructive deletes.

founder_admin

Rank: 4

Use for: Top-level founder authority, library/editorial work, health checks, account governance, and sensitive workflows.

Avoid using for: Routine content work when a lower role is enough; protect it like super_admin.

super_admin

Rank: 4

Use for: Top-level operations authority for settings, users, admin accounts, V-Vault, deletion, health checks, and sensitive operations.

Avoid using for: Routine content work when a lower role is enough.

Environment Variable Groups

Group

Variables | Why it matters

Public base URL

NEXT_PUBLIC_SITE_URL | Builds email links, metadata, sitemap, canonical URLs.

Database

DATABASE_URL, DIRECT_URL | Connects Prisma/CockroachDB and migration/deploy tooling.

Public sessions

JWT_SECRET | Signs public user JWT cookies.

Admin sessions/access

ADMIN_SESSION_SECRET, ADMIN*_ACCESS_CODE, ADMIN_ALLOWED_USERS, ADMIN_INVITE_TOKENS | Protects admin sign-in, registration roles, and invite-gated admin access.

Media storage

R2_ACCOUNT_ID, R2_ACCESS_KEY_ID, R2_SECRET_ACCESS_KEY, R2_BUCKET, R2_PUBLIC_URL | Enables media uploads and public asset URLs.

Email

RESEND_API_KEY, RESEND_FROM_EMAIL, RESEND_FROM_NAME | Enables verification and reset emails.

Feature flags

VAULT_SUBMISSIONS_ENABLED | Allows pausing public V-Vault submissions.

Data Model Quick Map

Identity

Models: User, AdminUser

Used by: Public auth, admin auth, account management.

Content

Models: Article, TeamMember, GalleryImage, PodcastEpisode, OutreachEvent, Review

Used by: Public publishing and admin content boards.

Engagement

Models: EventLike, EventComment, ReviewHelpful

Used by: Likes, comments, helpful marks.

Private support

Models: VaultSubmission, VaultResponse, VaultSubmissionEvent, UserNotification

Used by: V-Vault workflow and /me notifications.

Operations

Models: SiteSettings, SiteAlert, MediaAsset, AuditLog, ContactSubmission, RateLimitBucket

Used by: Settings, alerts, uploads, accountability, contact intake, and shared rate limiting.

Appendix E - Operator Quick Reference

Need to do

Go here | Remember

Publish an event

/admin/events | Draft first, stable slug, publish after checking stream and engagement.

Moderate event comments

/admin/events | Use visible/hidden/flagged.

Publish an article

/admin/library | Founder admin or higher; include disclaimer context and stable slug.

Upload media

/admin/media or upload modal | Allowed type/size, label, alt text; R2 must be configured.

Find why a backend flow is failing

/admin/health | Top-level admin only; check dependency status, route row, model probe, and Reference ID.

Approve or reply to V-Vault

/admin/vault | Editor+ can respond/moderate; top-level admin is required for identity reveal; response creates in-app notification.

Handle user reset

/forgot-password or admin user detail | Never ask for passwords. Use reset flow.

Create admin

/admin/register or /admin/admin-users | Use lowest role and verify email.

Suspend admin

/admin/admin-users | Top-level admin only; cannot suspend yourself; check whether notification email was sent.

Change public hero/footer

/admin/settings | Top-level admin only.

Post home alert

/admin/alerts | Short, scheduled, and remove when done.

Prepare deployment

Terminal and provider dashboard | Use pnpm verify:release, pnpm db:deploy, env checklist, smoke test.

Final Note to the DownBelow Team

The website is strongest when the team treats it as both a public trust surface and an internal operating system. Publish carefully. Keep sensitive workflows private. Use roles deliberately. Test important changes from the public side. When something fails, trace the flow: page, API, repository, database, email, storage. That simple path will solve most problems faster than guessing.

One Last Note from our Team Leader

Technology is only powerful when people feel confident using it.

We hope this book helps you take full ownership of the DownBelow website without fear. Explore it. Practice with it. Ask questions when needed. Make it yours.

Most importantly, keep using it to share hope, knowledge, and opportunity with the world.

Let us keep building impact together.

Bespoke Tech

Bespoke Technologies

Engineering the solutions for this, and the Next Generations_